UJAMAADAO - COMPLETE UPDATED BUSINESS LOGIC & SYSTEM MAP

Version: 2.0 - Post-Schema Audit (December 2025)

Purpose: Single source-of-truth describing what the system does, how data flows, constraints, edge-cases, and exact responsibilities each module must implement. Incorporates all decisions made during schema audit and architecture review.

Table of Contents

---

1. CORE CHANGES SUMMARY

What Changed vs Original Spec

Models REMOVED (Simplified System)

- **Petition/PetitionSignature** - Not in spec, use Proposals instead
- **ServiceRequest/ServiceRequestUpdate** - Redundant with EmergencyAlert
- **Poll/PollOption/PollVote** - Use Proposal system for decisions
- **Achievement/UserAchievement** - Impact Points is sufficient gamification
- **User.industryId** field - Replaced with many-to-many UserIndustry

Rationale: Keep system focused on core DAO functionality. These features add complexity without clear differentiation from existing features. Can be added later if users demand them.

Models ADDED (Critical Missing Features)

Authentication & Security (5 models)

- `EmailVerificationToken` - Magic link authentication
- `Session` - Device/session tracking
- `Account` - OAuth provider accounts
- `RecoveryRequest` - Account recovery flows
- `LoginEvent` - Security audit trail

User Management (6 models)

- `ResidenceChangeRequest` - 7-day ward move review
- `UserIndustry` - Many-to-many industry selection (max 3)
- `GoodsService` - Catalog of tradeable items per industry
- `UserGoodsService` - What user can provide/request
- `UserPrivacySettings` - Granular privacy controls
- `UserAccessibility` - Disability support preferences

Participation Rights (1 model)

- `ParticipationRightsLog` - Track PR earning/spending

Onboarding System (3 models)

- `OnboardingProgress` - Step-by-step tutorial tracking
- `OnboardingTutorial` - CMS for tutorial content
- `UserTutorialCompletion` - Individual tutorial progress
- `OnboardingMilestone` - First-time achievements

Economy & Work (4 models)

- `DuesPayment` - Monthly dues tracking

- `DuesPenalty` - Non-payment consequences
- `PhysicalWorkLog` - Document physical contributions
- `WorkVerification` - Peer/supervisor verification

System Configuration (1 model)

- `SystemConfiguration` - Store voting thresholds, IP formulas, rules

Total Added: 20 new models

Models UPDATED (Enhanced Existing Features)

Group Model - Two-Enum Architecture

prisma

```
// OLD (Confusing)
enum GroupType {
  WARD_COMMUNITY
  CONSTITUENCY_PROJECT  //  Implies only projects
  COUNTY_INITIATIVE  //  Implies campaigns
  BUSINESS
  // ... 40+ mixed types
}

// NEW (Clear Separation)
enum SystemGroupType {
  WARD
  CONSTITUENCY
  COUNTY
  NATIONAL
}

enum VoluntaryGroupType {
  BUSINESS_COLLECTIVE
  SAVINGS_CREDIT
  PROJECT_EXECUTION
  YOUTH_ORGANIZATION
  // ... 35+ voluntary types
}

model Group {
  isSystemGroup Boolean
  systemType  SystemGroupType?  // Only if isSystemGroup = true
```

```
  voluntaryType VoluntaryGroupType? // Only if isSystemGroup = false
  canBeDeleted  Boolean @default(true)  // System = false
  canBeRenamed  Boolean @default(true)  // System = false
}
```

Why This Matters:

- **Type-safe:** Database constraint prevents invalid states
- **Future-proof:** Add 100+ voluntary types without touching system types
- **Clear queries:** `where: { systemType: 'WARD' }` vs maintaining arrays
- **Better indexes:** Separate indexes for system vs voluntary

GroupMember Model - Auto-Enrollment Tracking

prisma

```
model GroupMember {
  // ... existing fields

  autoEnrolled Boolean @default(false)  // NEW: System membership
  canLeave     Boolean @default(true)   // NEW: System groups = false
}
```

Why This Matters:

- Users auto-join 4 system groups (ward → constituency → county → national)
- Cannot leave system groups (automatic membership)
- Can leave voluntary groups anytime

---

2. PARTICIPATION RIGHTS SYSTEM (NEW)

What Are Participation Rights (PR)?

PR = Renewable civic energy that enables platform actions

Think of PR as "action tokens" you earn and spend to participate in governance.

Key Principle:

- **Impact Points (IP)** = Permanent reputation (voting weight)
- **Participation Rights (PR)** = Renewable resource (action credits)
- **Utility Tokens (UT)** = Economic value (money)

How Users Earn PR

Action	PR Earned	Frequency	Notes
Pay dues (Ordinary)	+100 PR	Monthly	Primary PR source
Pay dues (Supporter)	+200 PR	Monthly	Higher tier
Pay dues (Sponsor)	+500 PR	Monthly	Premium tier
Wallet connection	+100 PR	Once	Onboarding bonus
Community verification	+50 PR	Once	Trust milestone
Attend ward meeting	+10 PR	Per event	Active participation
Complete educational module	+15 PR	Per module	Learning reward
Verify milestone (quality work)	+25 PR	Per milestone	Trusted verifier
Successful project execution	+50 PR	Per project	Major contribution

Monthly Regeneration (Automated Cron)

typescript

```
// Every 1st of month
- Active users (logged in last 30 days): +25 PR
- All users: PR capped at max (e.g., 500 PR)
```

How Users Spend PR

Action	PR Cost	Reasoning
Vote on proposal	-5 PR	Prevent vote spam
Create ward proposal	-50 PR	Prevent low-effort proposals
Create constituency proposal	-100 PR	Higher scope = higher stakes
Create county proposal	-150 PR	Even higher threshold
Create national proposal	-200 PR	Highest governance level
Create voluntary group	-100 PR	Prevent group spam
Submit residence change	-25 PR	Limit frequent moves
File dispute/conflict	-50 PR	Prevent frivolous cases
View/comment	0 PR	Always free

Non-Payment Consequences

Dues Overdue Penalties

Months Overdue	PR Impact	Voting Impact	Status	Restrictions
0 (Current)	0	100%	GOOD_STANDING	None
1 month	-50 PR	50% weight	GRACE_PERIOD	Warning emails sent
2 months	-100 PR	25% weight	LIMITED	Can't create proposals/groups
3+ months	All PR lost	0% (can't vote)	SUSPENDED	Read-only access

Grace Period Rules:

- 30-day grace period before penalties apply
- Email warnings at 15 days, 7 days, 1 day overdue
- Can resume full rights immediately upon payment
- Old penalties cleared after 1 month of good standing

PR Thresholds (Feature Gating)

typescript

```typescript
const PR_REQUIREMENTS = {
  VOTE: 5,                    // Need at least 5 PR to vote once
  CREATE_PROPOSAL_WARD: 50,
  CREATE_PROPOSAL_CONSTITUENCY: 100,
  CREATE_PROPOSAL_COUNTY: 150,
  CREATE_PROPOSAL_NATIONAL: 200,
  CREATE_GROUP: 100,
  MARKETPLACE_SELL: 20,       // Need active participation
  MILESTONE_VERIFICATION: 50, // Trusted verifiers only

  LOW_PR_WARNING: 20,         // Show warning to user
  SUSPENSION_THRESHOLD: 0,    // Account suspended
};
```

Database Schema

prisma

```prisma
model User {
  participationRights Int @default(0)
  lastPRReset DateTime?
  prWarningsSent Int @default(0)
  duesOverdueMonths Int @default(0)

  prLogs ParticipationRightsLog[]
}

model ParticipationRightsLog {
  id        String   @id @default(uuid())
  userId    String
```

```
  amount    Int      // Positive = earned, Negative = spent
  balance   Int      // PR balance after transaction
  reason    String   // "DUES_PAID", "VOTED", "CREATED_PROPOSAL"
  relatedEntityType String?
  relatedEntityId   String?
  metadata  Json?
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id])

  @@index([userId, createdAt])
}
```

Implementation Notes

Service Methods Needed:

typescript

```
// src/modules/economy/services/participationRights.service.ts

export const participationRightsService = {
  async awardPR(userId: string, amount: number, reason: string, metadata?: any) {
    // Add PR and create log entry
  },

  async spendPR(userId: string, amount: number, reason: string, metadata?: any) {
    // Deduct PR, throw error if insufficient
  },

  async checkPRRequirement(userId: string, requiredPR: number): boolean {
    // Return true if user has enough PR
  },

  async monthlyRegeneration() {
    // Cron job: regenerate PR for active users
  },

  async applyDuesPenalties() {
    // Cron job: check overdue dues and apply penalties
```

```
  },

  async getUserPRBalance(userId: string): number {
    // Get current PR balance
  },
};
```

---

3. SYSTEM GROUPS ARCHITECTURE (UPDATED)

What Are System Groups?

System Groups = Automatic geographic governance structures

Every user is automatically a member of 4 system groups based on their `primaryWard`:

```
User in "Kibera Ward" automatically joins:
├─ Kibera Ward Community (System Group)
├─ Lang'ata Constituency Community (System Group)
├─ Nairobi County Community (System Group)
└─ Kenya National Community (System Group)
```

System Groups Count

Level	Count	Type
Ward	1,450	SYSTEM_WARD
Constituency	290	SYSTEM_CONSTITUENCY
County	47	SYSTEM_COUNTY
National	1	SYSTEM_NATIONAL
TOTAL	**1,788**	System Groups

System vs Voluntary Groups

Feature	System Groups	Voluntary Groups
Creation	Auto-created by seed script	User-created
Membership	Automatic (based on ward)	User chooses to join
Can Leave	No (permanent membership)	Yes (anytime)
Can Delete	No (system managed)	Yes (by admin/creator)

Feature	System Groups	Voluntary Groups
Can Rename	No (official names)	Yes
Treasury	Yes (public funds)	Yes (member funds)
Proposals	Yes (location-scoped)	Yes (group-internal)
Types	4 types (Ward/Constituency/County/National)	35+ types (Business, Youth, etc.)

Database Schema

prisma

```
enum SystemGroupType {
  WARD
  CONSTITUENCY
  COUNTY
  NATIONAL
}

enum VoluntaryGroupType {
  BUSINESS_COLLECTIVE
  SAVINGS_CREDIT
  YOUTH_ORGANIZATION
  TECHNOLOGY_HUB
  // ... 35+ more types
}

model Group {
  id           String  @id
  name         String  @unique

  // Core distinction
  isSystemGroup Boolean @default(false)

  // Type-safe (mutually exclusive)
  systemType    SystemGroupType?
  voluntaryType VoluntaryGroupType?

  // System protections
  canBeDeleted  Boolean @default(true)
  canBeRenamed  Boolean @default(true)

  // Geographic scope
```

```
  locationScope LocationScope
  wardId        String?
  constituencyId String?
  countyId      String?

  // Validation constraint
  @@check(
    (isSystemGroup = true AND systemType IS NOT NULL AND voluntaryType IS NULL) OR
    (isSystemGroup = false AND voluntaryType IS NOT NULL AND systemType IS NULL)
  )
}

model GroupMember {
  id       String @id
  userId   String
  groupId  String
  role     GroupRole @default(MEMBER)

  // System membership
  autoEnrolled Boolean @default(false)  // True for system groups
  canLeave     Boolean @default(true)   // False for system groups

  joinedAt DateTime
  active   Boolean @default(true)
}
```

Auto-Enrollment Flow

On User Signup:

typescript

```typescript
async function enrollUserInSystemGroups(userId: string, primaryWardId: string) {
  // 1. Get user's geographic hierarchy
  const ward = await prisma.ward.findUnique({
    where: { id: primaryWardId },
    include: {
      constituency: { include: { county: true } }
    }
  });
```

```
// 2. Find all 4 system groups
const wardGroup = await prisma.group.findFirst({
  where: {
    systemType: 'WARD',
    wardId: ward.id
  }
});

const constituencyGroup = await prisma.group.findFirst({
  where: {
    systemType: 'CONSTITUENCY',
    constituencyId: ward.constituencyId
  }
});

const countyGroup = await prisma.group.findFirst({
  where: {
    systemType: 'COUNTY',
    countyId: ward.constituency.countyId
  }
});

const nationalGroup = await prisma.group.findFirst({
  where: { systemType: 'NATIONAL' }
});

// 3. Auto-enroll in all 4
const enrollments = [
  { groupId: wardGroup.id, level: 'WARD' },
  { groupId: constituencyGroup.id, level: 'CONSTITUENCY' },
  { groupId: countyGroup.id, level: 'COUNTY' },
  { groupId: nationalGroup.id, level: 'NATIONAL' },
];

for (const { groupId, level } of enrollments) {
  await prisma.groupMember.create({
    data: {
      userId,
      groupId,
```

```
            role: 'MEMBER',
            autoEnrolled: true,  // System membership
            canLeave: false,     // Cannot leave
            active: true,
          }
        });

        // Increment member count
        await prisma.group.update({
          where: { id: groupId },
          data: { memberCount: { increment: 1 } }
        });
      }

      // 4. Award IP for joining ward group
      await reputationService.awardPoints(userId, 50, 'JOINED_WARD_GROUP');
    }
```

On Residence Change (Approved):

typescript

```
async function updateSystemGroupMemberships(
  userId: string,
  oldWardId: string,
  newWardId: string
) {
  const oldWard = await getWardHierarchy(oldWardId);
  const newWard = await getWardHierarchy(newWardId);

  // Always leave old ward group
  await leaveSystemGroup(userId, oldWard.id, 'WARD');

  // If constituency changed
  if (oldWard.constituencyId !== newWard.constituencyId) {
    await leaveSystemGroup(userId, oldWard.constituencyId, 'CONSTITUENCY');
  }

  // If county changed
  if (oldWard.countyId !== newWard.countyId) {
```

```
    await leaveSystemGroup(userId, oldWard.countyId, 'COUNTY');
  }

  // Join new groups
  await joinSystemGroup(userId, newWard.id, 'WARD');

  if (oldWard.constituencyId !== newWard.constituencyId) {
    await joinSystemGroup(userId, newWard.constituencyId, 'CONSTITUENCY');
  }

  if (oldWard.countyId !== newWard.countyId) {
    await joinSystemGroup(userId, newWard.countyId, 'COUNTY');
  }

  // National group unchanged

  // IMPORTANT: Preserve old location's Impact Points
  // User keeps old ward IP in UserLocationImpact
  // New IP accrues to new ward
}
```

System Group Treasuries

Funding Sources:
1. **Monthly dues** - Users choose which treasury (ward/constituency/county)
2. **Parent treasury allocation** - County can allocate to constituencies
3. **Grants from higher levels** - National → County → Constituency → Ward
4. **Project surplus returns** - Completed projects return unused funds

Spending Rules:

Group Type	No Approval Needed	Group Approval	Formal Proposal Required
Ward	< KES 10,000	< KES 100,000	> KES 100,000
Constituency	< KES 50,000	< KES 500,000	> KES 500,000
County	< KES 100,000	< KES 1,000,000	> KES 1,000,000
National	KES 0	KES 0	All spending via proposal

Transparency Requirements:
- All transactions public (visible to all members)
- Monthly treasury reports auto-generated
- Annual external audits required
- Multi-sig required for large expenditures

UI Treatment

User's "My Groups" Page:
```
┌─────────────────────────────────────────────────┐
│ YOUR COMMUNITIES                                │
├─────────────────────────────────────────────────┤
│ SYSTEM GROUPS (Automatic Membership)            │
│                                                 │
│ Kibera Ward Community                           │
│ ├─ Type: Ward Community (System)                │
│ ├─ Members: 15,234                              │
│ ├─ Treasury: KES 2,450,000                      │
│ └─ [View Proposals] [View Treasury]             │
│                                                 │
│ Lang'ata Constituency Community                 │
│ ├─ Type: Constituency (System)                  │
│ ├─ Members: 142,500                             │
│ ├─ Treasury: KES 18,300,000                     │
│ └─ [View] [Cannot Leave]                        │
│                                                 │
│ Nairobi County Community                        │
│ ├─ Members: 4.5M                                │
│ └─ [View]                                       │
│                                                 │
│ Kenya National Community                        │
│ ├─ Members: 15M                                 │
│ └─ [View]                                       │
│                                                 │
├─────────────────────────────────────────────────┤
│ YOUR VOLUNTARY GROUPS (6)                       │
│                                                 │
│ Kibera Youth Tech Hub                           │
```

```
│       ├─ Type: Technology Hub                    │
│       ├─ Members: 45                             │
│       ├─ Role: MEMBER                            │
│       └─ [View] [Leave Group]                    │
│                                                  │
│      Nairobi Waste Management Initiative         │
│       ├─ Type: Environmental Group               │
│       ├─ Members: 230                            │
│       ├─ Role: LEADER                            │
│       └─ [Manage] [Leave Group]                  │
│                                                  │
│ [+ Create New Group]                             │
└──────────────────────────────────────────────────┘
```

Seed Script for System Groups

typescript

```typescript
// backend/prisma/seeds/systemGroups.seed.ts

export async function seedSystemGroups(
  wards: Ward[],
  constituencies: Constituency[],
  counties: County[]
) {
  console.log('    Creating system groups...');

  // 1. National Group (1)
  const nationalGroup = await prisma.group.upsert({
    where: { name: 'Kenya National Community' },
    update: {},
    create: {
      name: 'Kenya National Community',
      description: 'National-level governance and strategic decisions',
      isSystemGroup: true,
      systemType: 'NATIONAL',
      locationScope: 'NATIONAL',
      status: 'ACTIVE',
      canBeDeleted: false,
      canBeRenamed: false,
```

```
    },
  });

  // 2. County Groups (47)
  for (const county of counties) {
    await prisma.group.upsert({
      where: { name: `${county.name} County Community` },
      update: {},
      create: {
        name: `${county.name} County Community`,
        description: `County-level governance for ${county.name}`,
        isSystemGroup: true,
        systemType: 'COUNTY',
        locationScope: 'COUNTY',
        countyId: county.id,
        status: 'ACTIVE',
        canBeDeleted: false,
        canBeRenamed: false,
      },
    });
  }

  // 3. Constituency Groups (290)
  for (const constituency of constituencies) {
    await prisma.group.upsert({
      where: { name: `${constituency.name} Constituency Community` },
      update: {},
      create: {
        name: `${constituency.name} Constituency Community`,
        description: `Constituency-level projects for ${constituency.name}`,
        isSystemGroup: true,
        systemType: 'CONSTITUENCY',
        locationScope: 'CONSTITUENCY',
        constituencyId: constituency.id,
        countyId: constituency.countyId,
        status: 'ACTIVE',
        canBeDeleted: false,
        canBeRenamed: false,
      },
```

```
    });
  }

  // 4. Ward Groups (1,450)
  for (const ward of wards) {
    await prisma.group.upsert({
      where: { name: `${ward.name} Ward Community` },
      update: {},
      create: {
        name: `${ward.name} Ward Community`,
        description: `Local community group for ${ward.name} Ward`,
        isSystemGroup: true,
        systemType: 'WARD',
        locationScope: 'WARD',
        wardId: ward.id,
        constituencyId: ward.constituencyId,
        countyId: ward.countyId,
        status: 'ACTIVE',
        canBeDeleted: false,
        canBeRenamed: false,
      },
    });
  }

  console.log('   System groups created:');
  console.log('  - National: 1');
  console.log('  - Counties: 47');
  console.log('  - Constituencies: 290');
  console.log('  - Wards: 1,450');
  console.log('  - TOTAL: 1,788 system groups');
}
```

---

4. ONBOARDING SYSTEM (ENHANCED)

What Is Onboarding?

Onboarding = Progressive tutorial system that guides new users from signup to active participation

NOT just signup - it's the entire journey:
```
Registration → Learning the platform → Understanding governance → First contributions → Active member
```

Onboarding Phases

Phase 1: Account Setup (Required)

Step	Action	IP Reward	PR Reward	Status
1. Email signup	Create account	0	0	Required
2. Email verification	Click magic link	50	25	Required
3. Profile completion	Add name, phone, ward	25	0	Required
4. Select industries	Choose 1-3 industries	0	0	Required
5. Choose goods/services	Pick what you provide/need	0	0	Required
6. Privacy settings	Configure visibility	0	0	Optional
7. Accessibility	Set preferences	0	0	Optional

Subtotal: 75 IP, 25 PR (minimum to access platform)

Phase 2: Platform Education (Recommended)

Step	Action	IP	PR	Required For	Time
8. Platform intro	Learn basics	25	10	-	5 min
9. Governance tutorial	How voting works	25	10	**Voting**	10 min
10. Marketplace tutorial	How trading works	15	0	**Selling**	7 min
11. Projects tutorial	How execution works	15	0	-	8 min
12. Groups tutorial	How community works	15	0	-	6 min

Subtotal: 95 IP, 20 PR

Feature Gating:

- **Cannot vote** until Governance tutorial complete
- **Cannot sell** until Marketplace tutorial complete

Phase 3: Verification Steps (Optional but Recommended)

Step	Action	IP	PR	Unlocks
13. Phone verification	Verify via OTP/Mpesa	100	25	Higher trust, create proposals
14. Community verification	Verified by 3+ community members	200	50	Milestone verification, group creation

Step	Action	IP	PR	Unlocks
15. Location verification	Prove residence with docs	150	25	Local voting bonuses
16. Wallet connection	Connect Web3 wallet	200	100	Full governance, blockchain features

Subtotal: 650 IP, 200 PR

Phase 4: First Actions (Milestones)

Milestone	Action	IP	PR	Tracked In
Join ward group	Auto-enrolled	50	0	Auto on signup
Join voluntary group	Choose a group	50	10	OnboardingMilestone
Cast first vote	Vote on proposal	100	0	OnboardingMilestone
Attend first event	Check-in at event	75	10	OnboardingMilestone
Complete first module	Educational content	50	15	UserEducationalProgress
Create first listing	Marketplace item	25	0	OnboardingMilestone

Subtotal: 350 IP, 35 PR

Phase 5: Financial Participation (Critical)

Step	Action	IP	PR	Unlocks
Pay first month's dues	KES 60/200/1000	0	**100**	**Voting rights**

Critical Rule: Must pay dues to vote on formal proposals.

Total Onboarding Rewards

Completion Level	IP Earned	PR Earned	Features Unlocked
Minimum (Phase 1)	75	25	View platform, comment
Recommended (Phase 1-2)	170	45	Basic participation
Full Verification (Phase 1-3)	820	245	All features
Active Participant (All phases)	1,170+	380+	Maximum engagement

---

Database Schema

prisma

```
model OnboardingProgress {
  id     String @id
  userId String @unique

  // Phase 1: Account Setup
  emailVerified        Boolean @default(false)
  profileCompleted     Boolean @default(false)
```

industriesSelected Boolean @default(false)

goodsServicesSelected Boolean @default(false)

privacyConfigured Boolean @default(false)

// Phase 2: Tutorials

platformIntroCompleted Boolean @default(false) governanceTutorialCompleted Boolean @default(false) marketplaceTutorialCompleted Boolean @default(false) projectsTutorialCompleted Boolean @default(false) groupsTutorialCompleted Boolean @default(false)

// Phase 3: Verification phoneVerified Boolean @default(false) communityVerified Boolean @default(false) locationVerified Boolean @default(false) walletConnected Boolean @default(false)

// Phase 4: First Actions joinedWardGroup Boolean @default(false) joinedVoluntaryGroup Boolean @default(false) castFirstVote Boolean @default(false) attendedFirstEvent Boolean @default(false) completedFirstModule Boolean @default(false) createdFirstListing Boolean @default(false)

// Phase 5: Financial paidFirstDues Boolean @default(false)

// Metadata currentStep String @default("EMAIL_VERIFICATION") lastActiveStep String? totalIPEarned Int @default(0) totalPREarned Int @default(0)

startedAt DateTime @default(now()) completedAt DateTime? updatedAt DateTime @updatedAt }

model OnboardingTutorial { id String @id key String @unique // "platform_intro" title String description String content Json // Steps, images, videos, quiz category String // "BASICS", "GOVERNANCE", etc. order Int requiredFor String? // "VOTING", "MARKETPLACE_SELLING" ipReward Int @default(0) prReward Int @default(0) isOptional Boolean @default(true) estimatedMinutes Int @default(5) active Boolean @default(true)

completions UserTutorialCompletion[] }

model UserTutorialCompletion { id String @id userId String tutorialId String

completed Boolean @default(false) progress Int @default(0) // 0-100% skipped Boolean @default(false) ipEarned Int @default(0) prEarned Int @default(0) quizScore Int?

startedAt DateTime @default(now()) completedAt DateTime? lastViewedAt DateTime @default(now())

@@unique([userId, tutorialId]) }

model OnboardingMilestone { id String @id userId String milestoneKey String // "FIRST_VOTE", "FIRST_LISTING"

achieved Boolean @default(true) ipEarned Int prEarned Int @default(0) metadata Json? achievedAt DateTime @default(now())

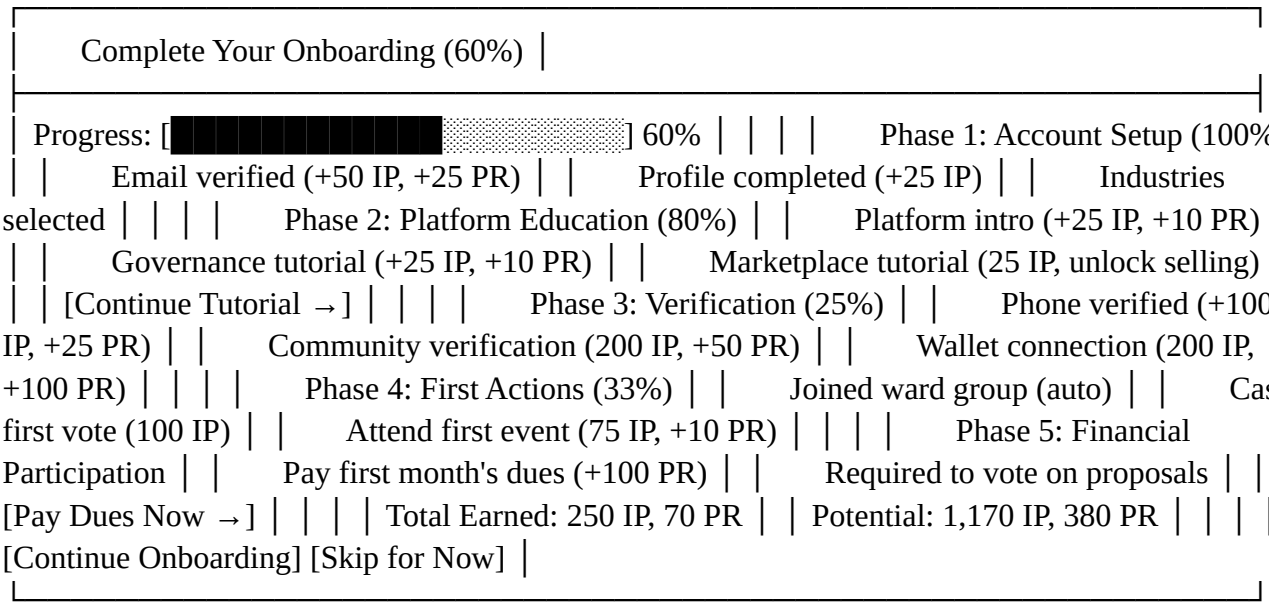
@@unique([userId, milestoneKey]) }

Service Methods

```typescript
// src/modules/onboarding/services/onboarding.service.ts

export const onboardingService = {
  async getProgress(userId: string) {
    const progress = await prisma.onboardingProgress.findUnique({
      where: { userId }
    });

    // Calculate completion percentage
    const totalSteps = 20; // Update based on actual steps
    const completedSteps = Object.values(progress).filter(v => v === true).length;
    const percentage = Math.round((completedSteps / totalSteps) * 100);

    return {
      progress,
      percentage,
      nextStep: determineNextStep(progress),
      unlockedFeatures: getUnlockedFeatures(progress),
    };
  },

  async completeStep(userId: string, step: string) {
    // Mark step complete
    await prisma.onboardingProgress.update({
      where: { userId },
      data: {
        [step]: true,
        lastActiveStep: step,
      }
    });

    // Award rewards if applicable
    const tutorial = await getTutorialForStep(step);
    if (tutorial) {
      await reputationService.awardPoints(userId, tutorial.ipReward, step);
      await participationRightsService.awardPR(userId, tutorial.prReward, step);
    }

    // Check if onboarding complete
```

```
    await checkOnboardingCompletion(userId);
  },

  async isFeatureUnlocked(userId: string, feature: string): boolean {
    const progress = await prisma.onboardingProgress.findUnique({
      where: { userId }
    });

    // Check feature requirements
    if (feature === 'VOTING') {
      return progress.governanceTutorialCompleted && progress.paidFirstDues;
    }

    if (feature === 'MARKETPLACE_SELLING') {
      return progress.marketplaceTutorialCompleted && progress.phoneVerified;
    }

    // ... more features
  },

  async recordMilestone(userId: string, milestoneKey: string, ipReward: number, prReward:
number) {
    await prisma.onboardingMilestone.create({
      data: {
        userId,
        milestoneKey,
        ipEarned: ipReward,
        prEarned: prReward,
      }
    });

    await reputationService.awardPoints(userId, ipReward, milestoneKey);
    await participationRightsService.awardPR(userId, prReward, milestoneKey);
  },
};
```

UI Example: Onboarding Dashboard



---

5. AUTHENTICATION & SESSION MANAGEMENT

Two-Phase Authentication System

Phase 1: Email Authentication (Passwordless)

- Magic link-based authentication
- No passwords to manage
- Secure, user-friendly
- Auto-login on email verification

Phase 2: Wallet Connection (Optional)

- Web3 wallet signature verification
- Unlocks full governance features
- Blockchain integration
- Higher trust level

Authentication Flow

New User Registration

1. User submits email + profile data

- ├─ Validate email format
- ├─ Check email not already registered
- ├─ Validate ward selection
- └─ Create User record (verificationLevel: UNVERIFIED)

2. Generate magic link token
   - ├─ Create EmailVerificationToken (24hr expiry)
   - ├─ Send email with verification link
   - └─ Return success message

3. User clicks magic link
   - ├─ Validate token exists and not expired
   - ├─ Update User (emailVerified: true, verificationLevel: EMAIL_VERIFIED)
   - ├─ Award 50 IP + 25 PR
   - ├─ Auto-enroll in 4 system groups
   - ├─ Create OnboardingProgress record
   - ├─ Generate JWT (180-day expiry)
   - └─ Auto-login to platform

Returning User Login

1. User enters email
   - ├─ Validate user exists
   - ├─ Check emailVerified = true
   - └─ Generate new magic link token (15min expiry)

2. Send login email
   - └─ Return "Check your email" message

3. User clicks login link
   - ├─ Validate token
   - ├─ Update lastLoginAt
   - ├─ Delete used token
   - ├─ Create LoginEvent record
   - ├─ Generate new JWT
   - └─ Redirect to dashboard

Wallet Connection Flow

```
1. User clicks "Connect Wallet" (must be authenticated)
    ├─ Check emailVerified = true
    ├─ Generate nonce
    ├─ Store nonce in User.nonce
    └─ Return signing message

2. User signs message with wallet
    └─ Message: "Sign this to connect your wallet\nNonce: {nonce}"

3. Server verifies signature
    ├─ Validate signature matches wallet address
    ├─ Check wallet not already connected to another account
    ├─ Update User (walletAddress, verificationLevel: FULL_VERIFIED)
    ├─ Award 200 IP + 100 PR
    ├─ Rotate nonce
    ├─ Update OnboardingProgress (walletConnected: true)
    ├─ Generate new JWT (with wallet claim)
    └─ Return success + new token
```

Database Schema

prisma

```
model EmailVerificationToken {
  id        String   @id @default(uuid())
  userId    String
  token     String   @unique
  expiresAt DateTime
  createdAt DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId])
  @@index([token, expiresAt])
}

model Session {
  id        String   @id @default(uuid())
  userId    String
```

```
  token      String   @unique
  deviceInfo String?
  ipAddress  String?
  userAgent  String?
  lastActive DateTime @default(now())
  expiresAt  DateTime
  revoked    Boolean  @default(false)
  createdAt  DateTime @default(now())
  updatedAt  DateTime @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, revoked])
  @@index([token, expiresAt])
}

model Account {
  id                String    @id @default(uuid())
  userId            String
  provider          String    // "google", "twitter"
  providerAccountId String
  accessToken       String?   @db.Text
  refreshToken      String?   @db.Text
  expiresAt         DateTime?
  tokenType         String?
  scope             String?
  idToken           String?   @db.Text
  createdAt         DateTime  @default(now())
  updatedAt         DateTime  @updatedAt

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@unique([provider, providerAccountId])
  @@index([userId])
}

model RecoveryRequest {
  id             String    @id @default(uuid())
  userId         String
  recoveryMethod   String    // "EMAIL", "WALLET", "RECOVERY_WALLET"
```

```
  verificationCode String    @unique
  status           String    @default("PENDING")
  expiresAt        DateTime
  completedAt      DateTime?
  ipAddress        String?
  userAgent        String?
  createdAt        DateTime  @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, status])
  @@index([verificationCode, expiresAt])
}

model LoginEvent {
  id           String   @id @default(uuid())
  userId       String
  method       String   // "MAGIC_LINK", "WALLET", "OAUTH"
  ipAddress    String?
  userAgent    String?
  location     String?
  successful   Boolean
  failureReason String?
  createdAt    DateTime @default(now())

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)

  @@index([userId, createdAt])
  @@index([successful, createdAt])
}
```

Security Measures

JWT Token Structure

typescript

```
interface JWTPayload {
  sub: string;              // userId
  email: string;
```

```
  phoneNumber?: string;
  walletAddress?: string;

  // Geographic context
  primaryWardId: string;
  constituencyId: string;
  countyId: string;

  // Verification levels
  verificationLevel: VerificationLevel;
  emailVerified: boolean;
  phoneVerified: boolean;
  communityVerified: boolean;
  locationVerified: boolean;

  // Roles & permissions
  roles: string[];

  // Economic data (for quick access)
  globalImpactPoints: number;
  utilityTokens: number;
  participationRights: number;

  // Token metadata
  type: "permanent" | "temporary";
  iat: number;
  exp: number;
}
```

Token Expiry Recommendations

Current Implementation:

- Access Token: 180 days (very long)

Recommended Implementation:

typescript

```
// Option A: Single long-lived token (current)
accessToken: "180d"  // Simple but less secure

// Option B: Access + Refresh pattern (recommended)
accessToken: "15m"   // Short-lived
```

```
refreshToken: "30d"  // Rotate on use
```

For MVP: Keep 180-day tokens (simplicity) **For Production: Implement refresh token rotation** (security)

Rate Limiting Rules

typescript

```
const RATE_LIMITS = {
  // Authentication endpoints
  'POST /auth/signup': {
    max: 5,
    window: '1h',
    message: 'Too many signup attempts'
  },

  'POST /auth/request-magic-link': {
    max: 3,
    window: '15m',
    message: 'Too many magic link requests'
  },

  'POST /auth/verify-email': {
    max: 10,
    window: '1h',
    message: 'Too many verification attempts'
  },

  'POST /auth/connect-wallet': {
    max: 5,
    window: '15m',
    message: 'Too many wallet connection attempts'
  },

  // General API
  'ALL /api/*': {
    max: 100,
    window: '15m',
    message: 'Too many requests'
  },
```

```
};
```

Session Management

typescript

```typescript
// Track active sessions per user
async function createSession(userId: string, deviceInfo: SessionInfo) {
  // Clean up expired sessions
  await prisma.session.deleteMany({
    where: {
      userId,
      expiresAt: { lt: new Date() }
    }
  });

  // Limit active sessions per user
  const activeSessions = await prisma.session.count({
    where: { userId, revoked: false }
  });

  if (activeSessions >= 5) {
    // Revoke oldest session
    const oldest = await prisma.session.findFirst({
      where: { userId, revoked: false },
      orderBy: { lastActive: 'asc' }
    });

    await prisma.session.update({
      where: { id: oldest.id },
      data: { revoked: true }
    });
  }

  // Create new session
  return await prisma.session.create({
    data: {
      userId,
      token: generateSecureToken(),
      deviceInfo: deviceInfo.userAgent,
```

```
        ipAddress: deviceInfo.ip,
        userAgent: deviceInfo.userAgent,
        expiresAt: new Date(Date.now() + 30 * 24 * 60 * 60 * 1000), // 30 days
      }
    });
}
```

Force Logout (tokenVersion)

typescript

```
// When user changes password, email, or roles
async function forceLogoutAllDevices(userId: string) {
  // Increment tokenVersion
  await prisma.user.update({
    where: { id: userId },
    data: { tokenVersion: { increment: 1 } }
  });

  // Revoke all sessions
  await prisma.session.updateMany({
    where: { userId },
    data: { revoked: true }
  });

  // All existing JWTs now invalid (check tokenVersion in middleware)
}

// Middleware check
async function validateJWT(token: string) {
  const decoded = jwt.verify(token, SECRET);
  const user = await prisma.user.findUnique({
    where: { id: decoded.sub },
    select: { tokenVersion: true }
  });

  if (!user || user.tokenVersion !== decoded.tokenVersion) {
    throw new Error('Token revoked');
  }
```

```
  return decoded;
}
```

Service Methods

typescript

```
// src/modules/auth/services/auth.service.ts

export const authService = {
  async signupWithEmail(data: SignupData) {
    // Create user + send magic link
  },

  async verifyEmail(token: string) {
    // Verify email + auto-enroll + award points + auto-login
  },

  async requestMagicLink(email: string) {
    // Generate token + send email
  },

  async loginWithMagicLink(token: string) {
    // Validate + create session + return JWT
  },

  async requestWalletNonce(walletAddress: string, userId: string) {
    // Generate nonce for wallet signature
  },

  async connectWallet(userId: string, walletAddress: string, signature: string) {
    // Verify signature + upgrade to FULL_VERIFIED + award points
  },

  async logout(userId: string, sessionToken?: string) {
    // Revoke session
  },

  async logoutAllDevices(userId: string) {
    // Force logout from all devices
```

```
  },
};
```

---

6. USER MANAGEMENT & PROFILE

User Profile Structure

Core User Fields

prisma

```
model User {
  id            String  @id
  walletAddress String? @unique
  email         String? @unique
  name          String?
  phoneNumber   String? @unique
  avatarUrl     String?

  // Geographic affiliations
  primaryWardId       String?
  secondaryWardId     String?
  currentLocationId   String?   // Temporary check-in
  currentLocationUntil DateTime?

  // Authentication
  nonce         String?
  tokenVersion  Int @default(0)

  // Verification status
  verificationLevel VerificationLevel @default(UNVERIFIED)
  emailVerified     Boolean @default(false)
  phoneVerified     Boolean @default(false)
  communityVerified Boolean @default(false)
  locationVerified  Boolean @default(false)

  // Account status
  isActive Boolean @default(true)

  // Economic data
```

```
  globalImpactPoints  Int @default(0)
  utilityTokens       Int @default(0)
  participationRights Int @default(0)
  lastPRReset         DateTime?
  prWarningsSent      Int @default(0)
  duesOverdueMonths   Int @default(0)

  // Recovery
  recoveryWallet String? @unique

  // Metadata
  lastLoginAt          DateTime?
  onboardingCompletedAt DateTime?
  metadata             Json?

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Industry & Goods/Services System

Business Rules

1. **Users can select 1-3 industries** (configurable: `maxIndustriesPerUser`)
2. **One industry must be marked primary**
3. **Goods/Services belong to specific industries**
4. **Users can only list goods/services from their selected industries**
5. **Prevent "jack-of-all-trades" abuse**

Schema

prisma

```
model Industry {
  id           String         @id
  name         String         @unique
  userIndustries UserIndustry[]
  goodsServices  GoodsService[]
}

model UserIndustry {
```

```
  id         String   @id
  userId     String
  industryId String
  isPrimary  Boolean  @default(false)
  createdAt  DateTime @default(now())

  user     User     @relation(fields: [userId], references: [id], onDelete: Cascade)
  industry Industry @relation(fields: [industryId], references: [id], onDelete: Cascade)

  @@unique([userId, industryId])
  @@index([userId])
}

model GoodsService {
  id          String   @id
  name        String
  description String?
  industryId  String
  category    String?
  active      Boolean  @default(true)

  industry  Industry          @relation(fields: [industryId], references: [id])
  userGoods UserGoodsService[]

  @@unique([name, industryId])
}

model UserGoodsService {
  id             String   @id
  userId         String
  goodsServiceId String
  canProvide     Boolean  @default(false)
  canRequest     Boolean  @default(false)
  createdAt      DateTime @default(now())

  user         User         @relation(fields: [userId], references: [id])
  goodsService GoodsService @relation(fields: [goodsServiceId], references: [id])

  @@unique([userId, goodsServiceId])
}
```

Example Data

typescript

```typescript
// Seed industries
const industries = [
  { name: "Agriculture" },
  { name: "Construction" },
  { name: "Technology" },
  { name: "Healthcare" },
  { name: "Education" },
  { name: "Transportation" },
  { name: "Retail" },
  { name: "Hospitality" },
];

// Seed goods/services per industry
const goodsServices = [
  // Agriculture
  { name: "Fresh Vegetables", industryId: "agriculture", category: "Produce" },
  { name: "Livestock", industryId: "agriculture", category: "Animals" },
  { name: "Farm Equipment Rental", industryId: "agriculture", category: "Services" },

  // Construction
  { name: "Cement", industryId: "construction", category: "Materials" },
  { name: "Labor Services", industryId: "construction", category: "Services" },
  { name: "Carpentry", industryId: "construction", category: "Skills" },

  // Technology
  { name: "Web Development", industryId: "technology", category: "Services" },
  { name: "Computer Repair", industryId: "technology", category: "Services" },
  { name: "Used Electronics", industryId: "technology", category: "Products" },
];
```

Validation Logic

typescript

```typescript
// When user selects industries
```

```
async function selectUserIndustries(userId: string, industryIds: string[], primaryIndustryId:
string) {
  const MAX_INDUSTRIES = 3;

  // Validate count
  if (industryIds.length > MAX_INDUSTRIES) {
    throw ApiError.validationError(`Maximum ${MAX_INDUSTRIES} industries allowed`);
  }

  if (industryIds.length === 0) {
    throw ApiError.validationError('At least one industry required');
  }

  // Validate primary is in list
  if (!industryIds.includes(primaryIndustryId)) {
    throw ApiError.validationError('Primary industry must be in selected industries');
  }

  // Delete existing
  await prisma.userIndustry.deleteMany({ where: { userId } });

  // Create new
  await prisma.userIndustry.createMany({
    data: industryIds.map(industryId => ({
      userId,
      industryId,
      isPrimary: industryId === primaryIndustryId,
    }))
  });

  // Update onboarding
  await prisma.onboardingProgress.update({
    where: { userId },
    data: { industriesSelected: true }
  });
}

// When user creates marketplace listing
async function validateListingIndustry(userId: string, goodsServiceId: string) {
  // Get goods/service industry
```

```
  const goodsService = await prisma.goodsService.findUnique({
    where: { id: goodsServiceId },
    select: { industryId: true }
  });

  // Check user has selected this industry
  const userIndustry = await prisma.userIndustry.findUnique({
    where: {
      userId_industryId: {
        userId,
        industryId: goodsService.industryId
      }
    }
  });

  if (!userIndustry) {
    throw ApiError.validationError(
      'You can only list goods/services from your selected industries'
    );
  }
}
```

Residence Management

Primary Ward Rules

1. **Every user has one primary ward** (required at signup)
2. **Primary ward determines system group membership**
3. **Primary ward is where impact points accrue by default**
4. **Changing primary ward requires verification (7-day review)**

Temporary Check-In System

Use Case: User travels to different ward for work/visit

typescript

```
async function checkIn(userId: string, wardId: string, days: number) {
  // Validate
  if (days < 1 || days > 90) {
    throw ApiError.validationError('Days must be between 1 and 90');
```

```
  }

  const user = await prisma.user.findUnique({
    where: { id: userId },
    select: { currentLocationId: true, primaryWardId: true }
  });

  // Cannot check in if already checked in
  if (user.currentLocationId) {
    throw ApiError.conflict('Already checked in elsewhere. Check out first.');
  }

  // Cannot check in to primary ward
  if (user.primaryWardId === wardId) {
    throw ApiError.validationError('You cannot check in to your primary ward');
  }

  const until = new Date(Date.now() + days * 24 * 60 * 60 * 1000);

  // Update user
  await prisma.user.update({
    where: { id: userId },
    data: {
      currentLocationId: wardId,
      currentLocationUntil: until,
    }
  });

  // Award temporary check-in points to THAT ward
  await locationImpactService.awardWardPoints(
    userId,
    wardId,
    25,
    'TEMPORARY_CHECKIN'
  );

  eventBus.publish('user.checked.in', { userId, wardId, days, until });

  return { message: 'Checked in successfully', until };
}
```

```
async function checkOut(userId: string) {
  await prisma.user.update({
    where: { id: userId },
    data: {
      currentLocationId: null,
      currentLocationUntil: null,
    }
  });

  eventBus.publish('user.checked.out', { userId });
}

// Automated check-out (cron job)
async function autoCheckOut() {
  const now = new Date();

  const expiredCheckIns = await prisma.user.findMany({
    where: {
      currentLocationId: { not: null },
      currentLocationUntil: { lte: now }
    }
  });

  for (const user of expiredCheckIns) {
    await checkOut(user.id);
  }
}
```

Permanent Residence Change

Use Case: User permanently moves to new ward

prisma

```
model ResidenceChangeRequest {
  id              String              @id
  userId          String
  oldWardId       String?
  newPrimaryWardId String
  proofUrl        String    // Document proof of residence
```

```
  status          ResidenceChangeStatus @default(PENDING)
  reviewedBy      String?
  reviewedAt      DateTime?
  rejectionReason String?
  expiresAt       DateTime  // Auto-reject after 7 days
  createdAt       DateTime  @default(now())

  user     User @relation("UserResidenceRequests", fields: [userId])
  reviewer User? @relation("ResidenceRequestReviewer", fields: [reviewedBy])
  newWard  Ward @relation("NewWardRequests", fields: [newPrimaryWardId])
  oldWard  Ward? @relation("OldWardRequests", fields: [oldWardId])

  @@index([userId, status])
  @@index([status, expiresAt])
}

enum ResidenceChangeStatus {
  PENDING
  APPROVED
  REJECTED
  EXPIRED
}
```

Flow:

typescript

```
async function requestResidenceChange(userId: string, newWardId: string, proofUrl: string) {
  // Check no pending request
  const pending = await prisma.residenceChangeRequest.findFirst({
    where: { userId, status: 'PENDING' }
  });

  if (pending) {
    throw ApiError.conflict('You already have a pending residence change request');
  }

  const user = await prisma.user.findUnique({
    where: { id: userId },
    select: { primaryWardId: true }
  });
```

```
  // Create request
  const request = await prisma.residenceChangeRequest.create({
    data: {
      userId,
      oldWardId: user.primaryWardId,
      newPrimaryWardId: newWardId,
      proofUrl,
      status: 'PENDING',
      expiresAt: new Date(Date.now() + 7 * 24 * 60 * 60 * 1000), // 7 days
    }
  });

  // Spend PR for request
  await participationRightsService.spendPR(userId, 25, 'RESIDENCE_CHANGE_REQUEST');

  // Notify ward admins for review
  await notificationService.notifyWardAdmins(newWardId, {
    type: 'RESIDENCE_CHANGE_REQUEST',
    userId,
    requestId: request.id,
  });

  return request;
}

async function approveResidenceChange(requestId: string, reviewerId: string) {
  const request = await prisma.residenceChangeRequest.findUnique({
    where: { id: requestId },
    include: { user: true }
  });

  if (request.status !== 'PENDING') {
    throw ApiError.conflict('Request already processed');
  }

  // Update request
  await prisma.residenceChangeRequest.update({
    where: { id: requestId },
    data: {
      status: 'APPROVED',
```

```
            reviewedBy: reviewerId,
            reviewedAt: new Date(),
        }
    });

    // Update user's primary ward
    await prisma.user.update({
        where: { id: request.userId },
        data: { primaryWardId: request.newPrimaryWardId }
    });

    // Update system group memberships
    await updateSystemGroupMemberships(
        request.userId,
        request.oldWardId,
        request.newPrimaryWardId
    );

    // IMPORTANT: Preserve old ward IP in UserLocationImpact
    // New IP will accrue to new ward, but old IP remains in history

    eventBus.publish('user.residence.changed', {
        userId: request.userId,
        oldWardId: request.oldWardId,
        newWardId: request.newPrimaryWardId,
    });
}
```

Privacy & Accessibility

Privacy Settings

prisma

```
model UserPrivacySettings {
    id                  String  @id
    userId              String  @unique

    profileVisibility   String  @default("PUBLIC") // PUBLIC, FRIENDS, PRIVATE
    showEmail           Boolean @default(false)
```

```
  showPhone            Boolean @default(false)
  showWallet           Boolean @default(false)
  showImpactPoints     Boolean @default(true)
  allowDirectMessages  Boolean @default(true)
  allowMarketplace     Boolean @default(true)
  dataProcessingConsent Boolean @default(false)

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Privacy Enforcement:

typescript

```typescript
// When returning user profile to API
async function getUserProfile(requesterId: string, targetUserId: string) {
  const user = await prisma.user.findUnique({
    where: { id: targetUserId },
    include: { privacySettings: true }
  });

  const settings = user.privacySettings;

  // Check visibility
  if (settings.profileVisibility === 'PRIVATE' && requesterId !== targetUserId) {
    throw ApiError.forbidden('Profile is private');
  }

  // Filter fields based on settings
  return {
    id: user.id,
    name: user.name,
    avatarUrl: user.avatarUrl,
    email: settings.showEmail ? user.email : null,
    phoneNumber: settings.showPhone ? user.phoneNumber : null,
    walletAddress: settings.showWallet ? user.walletAddress : null,
    globalImpactPoints: settings.showImpactPoints ? user.globalImpactPoints : null,
    // ... other fields
  };
}
```

Accessibility Settings

prisma

```
model UserAccessibility {
  id                String  @id
  userId            String  @unique

  visualImpairment    Boolean @default(false)
  hearingImpairment   Boolean @default(false)
  motorImpairment     Boolean @default(false)
  cognitiveImpairment Boolean @default(false)

  preferredLanguage   String  @default("en") // en, sw (Swahili)
  screenReaderEnabled Boolean @default(false)
  highContrast        Boolean @default(false)
  fontSize            String  @default("MEDIUM") // SMALL, MEDIUM, LARGE, XLARGE

  createdAt DateTime @default(now())
  updatedAt DateTime @updatedAt
}
```

Frontend Impact:
- Auto-adjust UI based on accessibility settings
- Provide audio alternatives for screen readers
- High contrast themes
- Keyboard navigation support
- Swahili language option

---

7. GEOGRAPHIC & RBAC

Geographic Hierarchy

```
Kenya (National)
├─ County (47 counties)
│   ├─ Constituency (~290 total)
│   │   ├─ Ward (~1,450 total)
```

Schema

prisma

```
model County {
  id            String         @id
  code          String         @unique
  name          String         @unique
  constituencies Constituency[]
  wards         Ward[]
  groups        Group[]
}

model Constituency {
  id       String @id
  name     String
  countyId String
  county   County @relation(fields: [countyId], references: [id], onDelete: Cascade)
  wards    Ward[]
  groups   Group[]

  @@unique([name, countyId])
  @@index([countyId])
}

model Ward {
  id            String       @id
  name          String
  code          String?      @unique
  constituencyId String
  countyId      String

  constituency Constituency @relation(fields: [constituencyId], references: [id])
  county       County       @relation(fields: [countyId], references: [id])

  // Relations
  groups              Group[]
  primaryUsers        User[]  @relation("PrimaryWard")
  secondaryUsers      User[]  @relation("SecondaryWard")
  currentLocationUsers  User[]  @relation("CurrentLocation")
  locationImpacts     UserLocationImpact[]
  // ... more relations
```

```
  @@index([constituencyId])
  @@index([countyId])
}
```

RBAC (Role-Based Access Control)

Role Structure

Namespaces:

- `system:*` - Platform-wide roles (super_admin, auditor)
- `location:*` - Geographic roles (ward_admin, county_admin)
- `group:*` - Group-specific roles (treasurer, leader)

Schema

prisma

```
model Role {
  id          Int      @id @default(autoincrement())
  name        String   @unique
  namespace   String?  // "system", "location", "group"
  builtin     Boolean  @default(false)
  description String?

  userRoles       UserRole[]
  rolePermissions RolePermission[]
}

model UserRole {
  id         String    @id
  userId     String
  roleId     Int
  scope      String?   // "ward:123", "group:456", null (global)
  active     Boolean   @default(true)
  assignedBy String?
  assignedAt DateTime  @default(now())
  expiresAt  DateTime?

  user User @relation(fields: [userId], references: [id], onDelete: Cascade)
```

```
  role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)

  @@unique([userId, roleId, scope])
  @@index([userId, active])
  @@index([roleId, scope])
}

model RolePermission {
  id          String @id
  roleId      Int
  permission  String  // "proposal:create", "project:approve_milestone"
  description String?

  role Role @relation(fields: [roleId], references: [id], onDelete: Cascade)

  @@unique([roleId, permission])
  @@index([permission])
}
```

Default Roles (Seed Data)

typescript

```
const DEFAULT_ROLES = [
  // System roles
  {
    name: 'system:super_admin',
    namespace: 'system',
    builtin: true,
    description: 'Full platform access',
    permissions: ['*:*'] // All permissions

},

{ name: 'system:auditor', namespace: 'system', builtin: true, permissions: ['audit:read',
'user:read', 'group:read', 'treasury:read'] },

// Location roles { name: 'location:ward_admin', namespace: 'location', builtin: true,
permissions: [ 'proposal:create:ward', 'proposal:moderate:ward', 'user:verify:ward',
'residence_request:review', ] },

{ name: 'location:county_admin', namespace: 'location', builtin: true, permissions:
[ 'proposal:create:county', 'treasury:manage:county', 'group:create:county', ] },
```

```
// Group roles { name: 'group:treasurer', namespace: 'group', builtin: true, permissions:
[ 'treasury:view', 'treasury:spend:small', // < threshold 'transaction:create', ] },

{ name: 'group:leader', namespace: 'group', builtin: true, permissions: [ 'group:manage',
'member:add', 'member:remove', 'proposal:create:group', ] }, ];
```

Permission Checking

```typescript
// Check if user has permission
async function userHasPermission(
    userId: string,
    permission: string,
    scope?: string
): Promise<boolean> {
    const userRoles = await prisma.userRole.findMany({
        where: {
            userId,
            active: true,
            OR: [
                { scope: scope },    // Specific scope
                { scope: null },     // Global role
            ]
        },
        include: {
            role: {
                include: {
                    rolePermissions: true
                }
            }
        }
    });

    // Check if any role grants permission
    for (const userRole of userRoles) {
        for (const rolePerm of userRole.role.rolePermissions) {
            if (matchesPermission(rolePerm.permission, permission)) {
                return true;
            }
        }
```

```
    }

    return false;
}

function matchesPermission(granted: string, required: string): boolean {
    // Wildcard support
    if (granted === '*:*') return true;
    if (granted === `${required.split(':')[0]}:*`) return true;
    return granted === required;
}

// Middleware usage
async function requirePermission(permission: string, scope?: string) {
    return async (req, res, next) => {
        const hasPermission = await userHasPermission(
            req.user.id,
            permission,
            scope
        );

        if (!hasPermission) {
            throw ApiError.forbidden(`Missing permission: ${permission}`);
        }

        next();
    };
}

// Route example
router.post(
    '/proposals/ward',
    authenticate,
    requirePermission('proposal:create:ward', req.user.primaryWardId),
    proposalController.create
);
```

8. GROUPS & COMMUNITY

Group Types & Architecture

Two-Enum System (Final Decision)

We use **two separate enums** for type-safety and future-proofing:

prisma

```
enum SystemGroupType {
  WARD
  CONSTITUENCY
  COUNTY
  NATIONAL
}

enum VoluntaryGroupType {
  // ECONOMIC (4 types)
  BUSINESS_COLLECTIVE
  SAVINGS_CREDIT
  AGRICULTURE_COOPERATIVE
  TRADE_ASSOCIATION

  // PROJECT (3 types)
  PROJECT_EXECUTION
  INFRASTRUCTURE_COMMITTEE
  CONSTRUCTION_TEAM

  // WELFARE (3 types)
  NON_PROFIT
  COMMUNITY_WELFARE
  POVERTY_ALLEVIATION

  // PROFESSIONAL (4 types)
  PROFESSIONAL_NETWORK
  EXPERT_PANEL
  QUALITY_ASSURANCE
  ACADEMIC_RESEARCH

  // EMERGENCY (4 types)
  EMERGENCY_RESPONSE
```

```
    FIRST_RESPONDERS
    SECURITY_WATCH
    CRISIS_MANAGEMENT

    // DEMOGRAPHIC (4 types)
    YOUTH_ORGANIZATION
    WOMEN_EMPOWERMENT
    ELDERLY_SUPPORT
    DISABILITY_ADVOCACY

    // SECTOR (6 types)
    HEALTH_INITIATIVE
    EDUCATION_INITIATIVE
    ENVIRONMENTAL_GROUP
    TECHNOLOGY_HUB
    ARTS_CULTURE
    SPORTS_RECREATION

    // FAITH & CULTURE (3 types)
    RELIGIOUS_ORGANIZATION
    CULTURAL_PRESERVATION
    INTERFAITH_DIALOGUE

    // ADVOCACY (4 types)
    ADVOCACY_GROUP
    CIVIC_EDUCATION
    POLICY_RESEARCH
    TRANSPARENCY_WATCHDOG

    // GENERAL (4 types)
    SPECIAL_INTEREST
    SOCIAL_CLUB
    ALUMNI_NETWORK
    NEIGHBORHOOD_ASSOCIATION
}
```

Why Two Enums?
- **Type-safe:** Database constraint prevents invalid states
- **Scalable:** Add 100+ voluntary types without touching system types

- **Clear queries:** No need to maintain arrays of "which types are system"
- **Better performance:** Separate indexes for each enum

Group Membership Rules

System Groups (Auto-Enrollment)

Rule	Description
Automatic membership	User auto-joins 4 system groups on signup
Cannot leave	System group membership is permanent
Based on primary ward	Ward → Constituency → County → National
Moves update membership	Residence change updates system groups

Voluntary Groups (User Choice)

Rule	Description
User-created	Any verified user can create
Join/leave freely	Users control their membership
Group types	35+ types for different purposes
Role-based permissions	Leader, Treasurer, Member, etc.

Group Treasury Management

All Groups Have Treasuries

Both system and voluntary groups can hold funds.

System Group Treasury Rules

Level	Small Spending (No Approval)	Medium (Group Approval)	Large (Formal Proposal)
Ward	< KES 10,000	KES 10k - 100k	> KES 100,000
Constituency	< KES 50,000	KES 50k - 500k	> KES 500,000
County	< KES 100,000	KES 100k - 1M	> KES 1,000,000
National	KES 0	KES 0	All spending via proposal

Voluntary Group Treasury Rules

- Groups set their own spending thresholds in constitution
- Multi-sig recommended for amounts > KES 50,000
- Treasurer role required for spending
- All transactions public to group members

Group Constitution

Each group can define:
- **Membership requirements** - Who can join
- **Voting rules** - Quorum, approval thresholds
- **Spending limits** - Treasury thresholds
- **Leadership terms** - How long leaders serve
- **Dissolution rules** - How group can be disbanded

Group Lifecycle

Formation Stage
```
1. User creates group (costs 100 PR)
    ├─ Select group type
    ├─ Set name, description
    ├─ Choose geographic scope
    └─ Draft constitution

2. Invite initial members
    └─ Minimum: 3 members for most types

3. Group becomes ACTIVE
    └─ Can create proposals, hold treasury
```

Active Operations

- Create internal proposals
- Manage treasury
- Execute projects
- Participate in marketplace

- Create events

Suspension/Dissolution

Suspension triggers:

- Inactivity > 6 months
- Financial irregularities
- Violations of platform rules

Dissolution process:

1. Group votes to dissolve (2/3 majority)
2. 30-day notice period
3. Treasury distributed per constitution
4. Assets transferred or liquidated
5. Group archived (not deleted - history preserved)

---

9. PROPOSALS & GOVERNANCE

Proposal Types & Tiers

Four Proposal Types

prisma

```
enum ProposalType {
  COMMUNITY_INITIATIVE  // Local ward improvements
  MAJOR_PROJECT         // Constituency/county infrastructure
  STRATEGIC_DECISION    // National policy changes
  EMERGENCY             // Fast-tracked urgent issues
}
```

Default Voting Thresholds

Stored in `SystemConfiguration` and adjustable:

Type	Quorum	Approval	Voting Period
Community Initiative	40%	50%	7 days
Major Project	50%	60%	14 days
Strategic Decision	60%	66%	21 days
Emergency	30%	60%	3 days

Voting Weight Calculation

Core Principle: Voting weight is determined by **Impact Points (IP)**, not roles.

Formula

typescript

```
votingWeight = f(globalIP, locationIP, tierMultiplier)

// Example implementation:
function calculateVotingWeight(
  userId: string,
  proposalScope: LocationScope,
  proposalLocationId: string
): number {
  const user = getUserImpact(userId);

  // Base weight from global IP
  let weight = user.globalImpactPoints;

  // Boost from location-specific IP
  if (proposalScope === 'WARD') {
    const wardIP = user.locationImpacts.find(l => l.wardId === proposalLocationId);
    weight += (wardIP?.impactPoints || 0) * 2; // 2x multiplier for local votes
  }

  // Tier multiplier (stored in SystemConfiguration)
  const tierMultiplier = getTierMultiplier(user.verificationLevel);
  weight *= tierMultiplier;

  return weight;
}
```

Location-Specific IP Boosts

- **Ward proposals:** Local ward IP counts 2x
- **Constituency proposals:** Constituency IP counts 1.5x
- **County proposals:** County IP counts 1.25x
- **National proposals:** Only global IP counts

Rationale: People most affected by decisions should have stronger voice.

Proposal Creation Requirements

Who Can Create Proposals?

Proposal Type	Minimum Verification	Minimum IP	PR Cost	Notes
Community (Ward)	PHONE_VERIFIED	100 IP	50 PR	Must be in that ward
Major (Constituency/County)	COMMUNITY_VERIFIED	500 IP	100-150 PR	Higher stakes
Strategic (National)	COMMUNITY_VERIFIED	1000 IP	200 PR	Platform-wide impact
Emergency	LOCATION_VERIFIED	200 IP	25 PR	Fast-track process

Proposal Scoping Rules

1. **Users can propose in their primary ward**
2. **System groups can propose one level up:**
   - Ward group → Can create ward OR constituency proposals
   - Constituency group → Can create constituency OR county proposals
   - County group → Can create county OR national proposals
3. **Voluntary groups propose based on their scope:**
   - If ward-scoped → Ward proposals only
   - If county-scoped → Ward, constituency, or county proposals

---

Proposal Funding Flow

Budget Allocation

```
1. Proposal created with budget estimate
   └─ Example: "Ward road repair: KES 500,000"

2. Voting period opens
   └─ Treasury funds LOCKED but not spent

3. Proposal APPROVED
   ├─ Funds allocated from specified treasury
   ├─ Project record created
```

```
    └─ Milestones defined with fund tranches

4. Milestone completion
    ├─ Verifiers approve work
    ├─ Tranche released to project wallet
    └─ On-chain transaction (if blockchain enabled)

5. Project completion
    ├─ Final audit
    ├─ Surplus returned to treasury
    └─ Impact recorded
```

⚠ Conflict of Interest Rules

Declaration Required When:

- Proposer is project beneficiary
- Proposer's family/business involved
- Proposer is group leader of affected group

Mitigation:

- Extra disclosure requirements
- Higher approval threshold (+10%)
- Cannot be milestone verifier for own proposal
- Independent audit required for large projects

Proposal States

```
DRAFT → User writing proposal
   ↓
VOTING → Open for community votes
   ↓
APPROVED → Passed quorum & approval
   ├→ EXECUTING → Project active
   │      └→ COMPLETED → Finished successfully
```

```
    |
    └→ REJECTED → Failed to meet thresholds
        └→ CANCELLED → Withdrawn by creator
```

Proposal Requirements

Minimum Information:

1. **Title** (clear, descriptive)
2. **Description** (detailed problem statement)
3. **Budget breakdown** (line-item costs)
4. **Timeline** (start/end dates, milestones)
5. **Impact metrics** (how success is measured)
6. **Beneficiaries** (who benefits)
7. **Risk assessment** (what could go wrong)

Optional Attachments:

- Design documents
- Cost estimates
- Community letters of support
- Technical feasibility studies

Vote Delegation (Future Feature)

Not in MVP but planned:

Users can delegate their voting power to trusted representatives.

typescript

```
// Delegation rules (future):
- Can delegate to one person per scope (ward/county/national)
- Can revoke delegation anytime
- Delegated votes count toward quorum
- Delegatee can't re-delegate (no chains)
```

---

10. PROJECTS, MILESTONES & PHYSICAL WORK

Project Execution Flow

From Proposal to Project

```
Approved Proposal
    ↓
Project Created
    ├─ Budget allocated
    ├─ Manager assigned
    ├─ Timeline set
    └─ Milestones defined
        ↓
Milestone 1: Site Preparation (20% of budget)
    ├─ Work performed
    ├─ Physical work logs submitted
    ├─ Photos/evidence uploaded
    ├─ Verifiers review
    └─ APPROVED → Funds released
        ↓
Milestone 2: Foundation (30% of budget)
    └─ ... repeat process
        ↓
Final Milestone: Completion (10% + surplus)
    ├─ Final audit
    ├─ Community acceptance
    └─ Project closed
```

Milestone Structure

Each Milestone Has:

prisma

```
model Milestone {
  id          String
  projectId   String
  title       String
  description String
  dueDate     DateTime?
  status      MilestoneStatus  // PENDING, UNDER_REVIEW, APPROVED, REJECTED
  budget      Decimal          // Fund tranche for this milestone
  completedAt DateTime?
```

```
  verifiedBy  String?          // Verifier user ID

  // Relations
  project  Project
  workLogs PhysicalWorkLog[]
}
```

Milestone Verification

Verification Requirements

Project Size	Required Verifiers	Verification Method
Small (< KES 100k)	1 verifier	Peer review
Medium (KES 100k-1M)	2 verifiers	Supervisor + Peer
Large (> KES 1M)	3+ verifiers	Expert panel

Verifier Selection Rules

1. **Must be independent** - Not project participant or beneficiary
2. **Must have verification credentials** - COMMUNITY_VERIFIED minimum
3. **Must have relevant expertise** - Technical knowledge for complex projects
4. **Cannot verify own work** - Conflict of interest prevention
5. **Earn PR for verification** - +25 PR per milestone verified

Verification Process
```
1. Milestone marked "complete" by project manager
   └─ Triggers notification to verifiers

2. Verifiers visit site / review evidence
   ├─ Check PhysicalWorkLogs
   ├─ Review photo evidence
   ├─ Inspect quality
   └─ Interview witnesses

3. Each verifier submits WorkVerification
   ├─ APPROVED - Work meets standards
```

├─ REJECTED - Work inadequate
└─ DISPUTED - Escalate to ConflictCase

4. If majority approve:
   ├─ Milestone status → APPROVED
   ├─ Funds released
   └─ IP awarded to workers

5. If rejected:
   ├─ Milestone status → REJECTED
   ├─ Funds held
   └─ Project manager notified to fix issues

Physical Work Logs

Purpose

Document physical contributions for:

- IP earning
- Milestone verification
- Project accountability
- Reputation building

Schema

prisma

```
model PhysicalWorkLog {
  id              String
  userId          String       // Worker
  projectId       String?
  milestoneId     String?
  workType        String       // MANUAL_LABOR, SKILLED_WORK, SUPERVISION
  description     String
  hours           Decimal      // Time spent
  location        Json?        // GPS coordinates
  photoUrls       String[]     // Evidence photos
  witnessIds      String[]     // Witness user IDs
  baseIP          Int          // Base IP for this work type
  qualityMultiplier Decimal    // 0.5 - 2.0 based on quality
  totalIPEarned   Int          // baseIP * qualityMultiplier * hours
```

```
  verifiedAt       DateTime?

  verifications WorkVerification[]
}
```

Evidence Requirements

Minimum Evidence for Work Logs:

1. **Photos** - Before, during, after
2. **Timestamp** - Automatic from upload
3. **Location** - GPS coordinates (optional but recommended)
4. **Witnesses** - At least 1 community witness
5. **Description** - Clear explanation of work done

Photo Validation:

typescript

```
// Automated checks:
- EXIF metadata verification (timestamp, location)
- Duplicate detection (prevent reusing old photos)
- Face detection (confirm people present)
- Timestamp within project timeline

// Manual review:
- Quality of work visible
- Work matches description
- Location matches project site
```

IP Earning from Physical Work

Base IP Rates (Configurable in SystemConfiguration)

Work Type	Base IP/Hour	Examples
Manual Labor	10 IP	Digging, carrying, cleaning
Skilled Work	20 IP	Carpentry, masonry, plumbing
Technical Work	30 IP	Engineering, surveying, design
Supervision	15 IP	Project oversight, quality control

Quality Multipliers

Applied by verifiers based on work quality:

Quality	Multiplier	Description
Poor	0.5x	Below standard, needs rework
Acceptable	1.0x	Meets basic requirements
Good	1.5x	Exceeds expectations
Excellent	2.0x	Outstanding craftsmanship

Example Calculation:
```
Worker: John
Work Type: Skilled carpentry
Hours: 8 hours
Base IP: 20 IP/hour
Quality: Good (1.5x multiplier)

Total IP = 20 * 8 * 1.5 = 240 IP
```

Disputed Work

When Disputes Arise:

1. Verifier rejects work log
2. Worker disagrees with rejection
3. ConflictCase created
4. Mediators assigned
5. Evidence reviewed
6. Ruling issued
7. IP awarded or denied based on ruling

---

11. ECONOMIC SYSTEM (IP, UT, PR, DUES)

Three Currencies Explained

Impact Points (IP) - Reputation

- **Non-transferable** - Cannot be bought or sold
- **Permanent** - Never removed (but can decay monthly)
- **Determines voting weight** - More IP = stronger voice
- **Earned through contribution** - Work, education, participation
- **Location-specific tracking** - IP per ward + global IP

Utility Tokens (UT) - Economic Value

- **Transferable** - Can be traded
- **Fungible** - All tokens equal value
- **Used for payments** - Marketplace, services, fees
- **Convertible to fiat** - Can cash out via Mpesa
- **Exchange rate** - 1 UT = KES X (configurable)

Participation Rights (PR) - Action Credits

- **Renewable** - Regenerates monthly
- **Consumed by actions** - Voting, proposals cost PR
- **Earned by participation** - Dues, contributions, activity
- **Gates features** - Need PR to create proposals, vote
- **Prevents spam** - Can't vote 1000 times without contributing

---

Complete Economic Flow

```
User Contribution (Work, Education, Dues)
    ↓
Earns IP + PR (+ sometimes UT)
    ↓
IP → Determines voting weight in proposals
PR → Enables creation of proposals, voting
UT → Spend in marketplace, pay for services
    ↓
Participation → Earns more IP/PR
Marketplace activity → Earns UT
```

Monthly Dues System

Dues Tiers

prisma

```
enum DuesTier {
  ORDINARY   // KES 60/month
  SUPPORTER  // KES 200/month
  SPONSOR    // KES 1000/month
}
```

What Dues Payment Provides:

1. **+100 PR** (Ordinary), +200 PR (Supporter), +500 PR (Sponsor)
2. **Voting rights** - Must have paid dues to vote on formal proposals
3. **Treasury support** - Funds go to chosen treasury (ward/constituency/county)
4. **Platform sustainability** - Keeps system running

Treasury Allocation Choice

When paying dues, user chooses where funds go:

```
User pays KES 60 (Ordinary tier)
  ↓
Choose treasury level:
  ├─ Ward → Funds go to ward system group treasury
  ├─ Constituency → Funds go to constituency treasury
  └─ County → Funds go to county treasury
```

Recommendation: Most users should fund their ward treasury (grassroots impact).

⚠ Non-Payment Consequences

Grace Period: 30 Days

No penalties for first 30 days after due date. Warnings sent at:
- 15 days overdue
- 7 days overdue

- 1 day overdue

Penalty Schedule

Months Overdue	PR Impact	Voting Weight	Status	Restrictions
0	0	100%	GOOD_STANDING	None
1 month	-50 PR	50%	GRACE_PERIOD	Warning emails
2 months	-100 PR	25%	LIMITED	Can't create proposals/groups
3+ months	All PR lost	0% (suspended)	SUSPENDED	Read-only access

Recovery:

- Pay overdue amount immediately restores rights
- 1 month of good standing clears penalties
- Can request hardship waiver (community decision)

IP Decay System

Purpose

Prevent stagnation - encourage ongoing participation.

Monthly Decay Rate

- **Default:** 10% per month (configurable)
- **Applied to:** All IP types (global + location)
- **Minimum floor:** Cannot decay below 0
- **Active user bonus:** Users who participated in last 30 days: decay reduced to 5%

Example:
```
Month 1: User has 1000 IP
    └─ No activity this month
    └─ Decay: 1000 * 0.10 = 100 IP lost
    └─ New balance: 900 IP

Month 2: User votes on 3 proposals
    └─ Active user bonus: 5% decay instead of 10%
    └─ Decay: 900 * 0.05 = 45 IP lost
```

    └─ New balance: 855 IP
```

### **Decay Exemptions:**

- First 3 months after signup (grace period)
- Users with disabilities (accessibility consideration)
- Users on approved leave of absence

---

##  IP Transfer Rules

### **IP is NON-TRANSFERABLE**

-  Cannot gift IP to others
-  Cannot sell IP
-  Cannot trade IP

**Why?** IP represents personal reputation and contribution, not economic value.

### **BUT: Location IP Follows User**

When user moves wards:
-  Old ward IP stays in UserLocationImpact (history preserved)
-  New IP accrues in new ward
-  Global IP unaffected

---

##  Wallet Integration

### **User Wallets** (WalletBalance)

- Holds KES (Kenyan Shillings)
- Linked to Mpesa for deposits/withdrawals
- Used for marketplace transactions
- Can receive project payments

### **Group Treasuries** (WalletBalance + TokenBalance)

- Hold both KES and UT
```

- Multi-sig for large amounts
- Public transaction ledger
- Monthly reports auto-generated

Blockchain Integration (Optional)

- UT can be minted as ERC-20 tokens
- On-chain governance for proposals
- Immutable treasury transactions
- Smart contract-controlled releases

Mpesa Integration

Deposit Flow:
```
1. User initiates deposit
   └─ Enter amount (KES)

2. Mpesa STK Push sent
   └─ User enters PIN on phone

3. Payment callback received
   └─ Verify mpesaReceipt

4. Credit user wallet
   └─ Create WalletTransaction record

5. Award any bonuses
   └─ First deposit: bonus IP
```

Withdrawal Flow:
```
1. User requests withdrawal
   ├─ Check balance sufficient
   ├─ Check daily withdrawal limit
   └─ Verify phone number

2. Initiate Mpesa B2C
```

└─ Send KES to user phone

3. Await confirmation
└─ Retry if failed

4. Update wallet balance
└─ Create WalletTransaction

5. Send confirmation SMS
```

### **Security Measures:**

- Daily withdrawal limits (configurable per user tier)
- Mpesa receipt verification (prevent fraud)
- Webhook signature validation
- Idempotency (prevent duplicate processing)
- Rate limiting on financial endpoints

---

# 12. MARKETPLACE

## Marketplace Rules (Critical)

### **Trading Restrictions**

Based on your specific requirements:

| Seller Type | Can Sell To | Can Buy From |
|---|---|---|
| **Individual** | Groups ONLY | Groups ONLY |
| **Group** | Groups + Individuals | Groups + Individuals |

**Simplified:**
- **NO individual-to-individual trading**
- **Individuals must trade through groups**
- **Groups can trade with anyone**

**Why?** Encourages group formation and community cohesion.

---
```

Industry-Based Listings

Rules:

1. **Users select 1-3 industries during onboarding**
2. **Can only list goods/services from selected industries**
3. **Goods/Services pre-seeded per industry**
4. **Prevents "jack-of-all-trades" spam**

Example:
```
User selects industries:
├─ Agriculture
├─ Technology
└─ (empty - can select 1 more)

Can list:
    Fresh vegetables (Agriculture)
    Computer repair (Technology)
    Web development (Technology)

Cannot list:
    Construction materials (not in user's industries)
    Medical services (not in user's industries)
```

Marketplace Transaction Flow

Standard Transaction:
```
1. Buyer browses listings
    └─ Filter by location, industry, price

2. Buyer initiates purchase
    ├─ Select quantity
    ├─ Choose payment method (UT or KES)
    └─ Funds moved to ESCROW
```

3. Seller accepts order
   └─ Delivery details agreed

4. Delivery happens
   ├─ Buyer receives goods/service
   └─ Can dispute if issues

5. Buyer confirms receipt
   ├─ Funds released from escrow to seller
   ├─ Transaction completed
   └─ Both parties can leave reviews

6. Dispute resolution (if needed)
   └─ Mediator reviews evidence
   └─ Ruling: refund or release funds

Review System

Review Requirements:

- **Verified purchase only** - Must have completed transaction
- **One review per transaction** - No duplicate reviews
- **Rating:** 1-5 stars
- **Comment:** Optional written feedback
- **Response:** Seller can respond once

Review Impact:

- Affects seller's `qualityRating` (average of all reviews)
- Low ratings (< 3.0) flag listing for review
- Consistently low ratings can suspend seller
- High ratings (> 4.5) badge seller as "Trusted"

Marketplace Safety

Fraud Prevention:

1. **Escrow system** - Funds held until delivery confirmed
2. **Verified sellers only** - PHONE_VERIFIED minimum
3. **Industry validation** - Can't list outside industries
4. **Photo requirements** - Listings need clear photos
5. **Dispute resolution** - ConflictCase for issues

Quality Control:

- Listings auto-expire after 90 days (must renew)
- Inactive sellers flagged
- Scam reports investigated
- Repeat offenders banned

---

13. EDUCATIONAL MODULES

Learning System

Purpose:

- Educate members on governance, skills, community development
- Award IP for learning
- Build capacity across the network
- Enable economic mobility

Module Structure

prisma

```
model EducationalModule {
  id            String
  title         String
  description   String
  content       String   // Rich text, videos, interactive
  mediaUrls     String[]  // External videos, PDFs
  duration      Int      // Minutes
  difficulty    EducationalContentLevel  // BEGINNER, INTERMEDIATE, ADVANCED, EXPERT
  category      String   // "Governance", "Agriculture", "Technology"
  creatorId     String
  verified      Boolean   // Platform reviewed
  expertApproved Boolean   // Expert panel approved
  completionIP  Int      // IP awarded on completion
  views         Int
  averageRating Decimal
}
```

Module Categories

1. **Governance** - How DAO works, voting, proposals
2. **Skills** - Technical skills (carpentry, coding, farming)
3. **Business** - Entrepreneurship, financial literacy
4. **Health** - First aid, nutrition, wellness
5. **Technology** - Digital literacy, mobile money
6. **Environment** - Sustainability, conservation
7. **Civic Education** - Rights, responsibilities, advocacy

Completion Requirements

For Each Module:

1. **Watch/read all content** - Progress tracked
2. **Pass assessment** (if included) - 70% minimum score
3. **Maximum 3 attempts** - Prevents gaming
4. **Time minimum** - Must spend at least X minutes (prevent rushing)

Upon Completion:

- Award `completionIP` (50-200 IP based on difficulty)
- Certificate generated (on-chain if blockchain enabled)
- Unlock next module in series
- Record in UserEducationalProgress

IP Rewards by Difficulty

Difficulty	Typical Duration	IP Reward	Example
BEGINNER	15-30 min	50 IP	"Intro to Voting"
INTERMEDIATE	30-60 min	100 IP	"Creating Proposals"
ADVANCED	1-2 hours	150 IP	"Project Management"
EXPERT	2+ hours	200 IP	"Smart Contract Development"

Content Creation

Who Can Create Modules?

- **Platform admins** - Official content
- **Expert panel members** - Subject matter experts
- **Verified users** - Community-created (requires approval)
- **Groups** - Group-specific training

Approval Process:
```
1. Creator drafts module
   └─ Must be COMMUNITY_VERIFIED minimum

2. Submit for review
   └─ Expert panel reviews

3. Feedback provided
   ├─ Approve → Module goes live
   ├─ Request changes → Creator revises
   └─ Reject → Not suitable

4. Live module
   └─ Users can enroll
   └─ Creator earns IP per completion (incentive)
```

Learning Analytics

Track at user level:

- Modules started
- Modules completed
- Total learning hours
- Skill certifications earned
- IP earned from education

Track at platform level:

- Popular modules
- Completion rates
- Average scores
- Drop-off points (improve content)

---

14. EMERGENCY RESPONSE

Emergency Alert System

Purpose:

Rapid coordination for disasters, crises, urgent community needs.

Alert Types

prisma

```
enum EmergencyType {
  NATURAL_DISASTER   // Floods, earthquakes, fires
  HEALTH_CRISIS      // Disease outbreaks, medical emergencies
  CONFLICT           // Violence, civil unrest
  INFRASTRUCTURE_FAILURE  // Water, power, roads
  ENVIRONMENTAL      // Pollution, chemical spills
}

enum EmergencySeverity {
  LOW       // Monitor situation
  MEDIUM    // Prepare response
  HIGH      // Active response needed
  CRITICAL  // Immediate all-hands response
}
```

Emergency Alert Flow

```
1. Verified user reports emergency
    ├─ Location
    ├─ Type
    ├─ Severity
    ├─ Description
    └─ Needed resources

2. Alert broadcast to:
    ├─ Ward system group
```

```
    ├─ Emergency response groups
    ├─ Platform admins
    └─ Nearby users (push notifications)

3. Volunteers respond
    ├─ Indicate availability
    ├─ Specify skills (medical, rescue, logistics)
    ├─ Offer resources (vehicles, supplies)
    └─ Tracked in EmergencyResponse

4. Coordination
    ├─ Emergency group assigned
    ├─ Resources allocated
    ├─ Tasks distributed
    └─ Real-time updates

5. Resolution
    ├─ Emergency marked resolved
    ├─ Debrief and lessons learned
    ├─ Award IP to responders
    └─ Archive for future reference
```

Emergency Groups

Special group type: `EMERGENCY_RESPONSE`

Characteristics:

- **Pre-trained members** - First aid, rescue skills
- **Always on call** - Get priority notifications
- **Equipment ready** - Medical kits, tools, vehicles
- **Fast mobilization** - Can respond within minutes
- **Earn higher IP** - Emergency response = 2x IP multiplier

Emergency Funding

Fast-Track Treasury Access:

For CRITICAL emergencies:

- Ward treasury: immediate release up to KES 50,000
- County treasury: up to KES 500,000 with 2-signature approval

- National: emergency fund (pre-allocated)

Accountability:

- All emergency spending audited within 7 days
- Receipts required
- Community review of expenses
- Penalties for misuse

False Emergency Prevention

Validation:

- Reporter must be PHONE_VERIFIED minimum
- Penalties for false alarms:
  - 1st offense: Warning
  - 2nd offense: Suspend emergency reporting (30 days)
  - 3rd offense: Platform suspension

Verification:

- Cross-reference with other reports
- Verify with local authorities
- Photo/video evidence encouraged
- Community members can confirm/dispute

---

15. NOTIFICATIONS & INTEGRATIONS

Notification System

Notification Types:

prisma

```
enum NotificationType {
  SYSTEM        // Platform updates, maintenance
  PROPOSAL      // New proposals, voting reminders
  VERIFICATION  // Verification requests, approvals
  ECONOMIC      // Payments, treasury changes
}

enum NotificationPriority {
  LOW       // General updates
  NORMAL    // Standard notifications
```

```
  HIGH      // Important actions needed
  URGENT    // Emergency alerts
}
```

Notification Channels

Users can configure per channel:

Channel	Use Case	Settings
In-App	All notifications	Always on
Email	Daily digest or instant	Frequency choice
SMS	URGENT only	Number verification required
Push	Mobile app	Permission-based

Notification Preferences

prisma

```
model NotificationPreference {
  id       String
  userId   String
  channel  String  // "EMAIL", "SMS", "PUSH", "IN_APP"
  category String  // "PROPOSALS", "TREASURY", "EMERGENCY"
  enabled  Boolean @default(true)
}
```

Example User Settings:

```
- In-app: All categories enabled
- Email: Proposals + Emergency only (daily digest)
- SMS: Emergency only (instant)
- Push: Proposals + Treasury (instant)
```

Mpesa Integration

Use Cases:

1. **Dues Payment** - Monthly contributions
2. **Wallet Deposits** - Add funds to platform wallet
3. **Withdrawals** - Cash out earnings

4. **Marketplace Payments** - Buy goods/services
5. **Phone Verification** - Use billing as verification

Security Requirements:

- **Webhook signatures** - Validate callback authenticity
- **Idempotency keys** - Prevent duplicate transactions
- **Receipt validation** - Verify Mpesa codes
- **Rate limiting** - Prevent payment spam
- **Test mode** - Use sandbox for development

Banking Integration (Future)

Planned:

- Bank account deposits/withdrawals
- International transfers (for diaspora)
- Bulk payouts for project workers
- Treasury interest earning

Not in MVP.

External Messaging

Important Decision:

We will NOT build internal chat.

Instead:

- Provide Telegram/WhatsApp group links for groups
- In-app: Comments on proposals, projects, listings
- Use email/SMS for transactional messages
- Emergency alerts via push notifications

Why?

- Chat is complex (moderation, storage, real-time)
- External tools are mature (Telegram, WhatsApp)
- Focus resources on core DAO features
- Can always add later if needed

---

16. AUDITING, PRIVACY & CONSENT

Audit Trail

What We Log:

prisma

```
model UserAudit {
  id        String
  userId    String
  action    String    // "CREATED_PROPOSAL", "VOTED", "PAID_DUES"
  details   Json?     // Additional context
  ip        String?
  userAgent String?
  createdAt DateTime
}
```

Key Actions Logged:

- Authentication (login, logout, wallet connect)
- Financial (payments, withdrawals, transfers)
- Governance (proposals, votes)
- Administrative (role changes, suspensions)
- Sensitive data access (viewing private profiles)

Data Privacy

GDPR-Style Compliance:

Even though Kenya's data protection laws differ, we follow best practices:

1. **Consent tracking** - UserConsent model
2. **Data minimization** - Only collect what's needed
3. **Right to access** - Users can download their data
4. **Right to erasure** - Users can request deletion
5. **Data portability** - Export in standard format
6. **Breach notification** - Alert users within 72 hours

Consent Management

prisma

```
model UserConsent {
```

```
  id         String
  userId     String
  policyKey  String    // "TERMS_OF_SERVICE", "PRIVACY_POLICY", "DATA_PROCESSING"
  accepted   Boolean
  version    String    // Policy version number
  acceptedAt DateTime?
  ip         String?
  userAgent  String?
}
```

When Policies Update:
```
1. Policy version increments
   └─ Example: v1.0 → v1.1

2. Users see banner on next login
   └─ "We've updated our Privacy Policy"

3. User must accept to continue
   └─ New UserConsent record created

4. Track who accepted which version
   └─ Legal compliance
```

Data Deletion Requests

Process:
```
1. User requests deletion
   └─ Account → Settings → Delete Account

2. Pre-deletion checks
   ├─ Active projects? → Transfer or complete first
   ├─ Treasury funds? → Withdraw first
   ├─ Group leadership? → Transfer role first
   └─ Pending proposals? → Withdraw or wait for completion
```

3. Grace period (30 days)
   └─ Can cancel deletion request

4. Anonymization (not full deletion)
   ├─ PII removed (name, email, phone, wallet)
   ├─ Contributions preserved (IP logs, votes)
   ├─ User ID remains (audit trail integrity)
   └─ Account marked: `isActive: false, deletedAt: now()`

5. Notify user confirmation
```

**Why not full deletion?**
- Governance records must be immutable
- Financial audit requirements
- Legal obligations
- Maintain platform integrity

---

## PII Handling

### **Personally Identifiable Information:**

- Email (encrypted at rest)
- Phone number (hashed for lookups)
- Wallet address (public by nature)
- IP addresses (anonymized after 90 days)
- Photos (face detection, consent required)

### **Storage:**

- Sensitive data encrypted (AES-256)
- Backups encrypted
- Access logs for PII views
- Automatic PII redaction in logs

---

# 17. COMPLETE SCHEMA STRUCTURE

## File Organization
```

```
backend/
├── prisma/
│   ├── schema.prisma         # Generated (don't edit)
│   ├── base.prisma           # Core shared models
│   └── modules/
│       ├── auth.prisma       # Authentication
│       ├── user.prisma       # User management
│       ├── economy.prisma    # Dues, work, PR
│       ├── community.prisma  # Groups, assets
│       ├── governance.prisma # Proposals, votes
│       ├── marketplace.prisma # Trading
│       ├── education.prisma  # Learning
│       ├── emergency.prisma  # Alerts
│       └── audit.prisma      # Logging, privacy
```

Merge Script

typescript

```typescript
// backend/src/core/database/mergeSchema.ts

import * as fs from 'fs';
import * as path from 'path';

const MODULE_ORDER = [
  'base.prisma',
  'modules/auth.prisma',
  'modules/user.prisma',
  'modules/economy.prisma',
  'modules/community.prisma',
  'modules/governance.prisma',
  'modules/marketplace.prisma',
  'modules/education.prisma',
  'modules/emergency.prisma',
  'modules/audit.prisma',
];

export function mergeSchemas() {
  const prismaDir = path.join(__dirname, '../../../prisma');
```

```
    const outputPath = path.join(prismaDir, 'schema.prisma');

    let merged = '';
    const seenModels = new Set<string>();

    for (const file of MODULE_ORDER) {
        const filePath = path.join(prismaDir, file);
        const content = fs.readFileSync(filePath, 'utf8');

        // Extract model names
        const modelMatches = content.matchAll(/model\s+(\w+)\s+{/g);
        for (const match of modelMatches) {
            const modelName = match[1];
            if (seenModels.has(modelName)) {
                throw new Error(`Duplicate model: ${modelName} in ${file}`);
            }
            seenModels.add(modelName);
        }

        merged += `\n//
==========================================\n`;
        merged += `// FROM: ${file}\n`;
        merged += `// ==========================================\
n\n`;
        merged += content + '\n';
    }

    fs.writeFileSync(outputPath, merged);
    console.log('    Schema merged successfully');
    console.log(`   Models: ${seenModels.size}`);
}

if (require.main === module) {
    mergeSchemas();
}
```

NPM Scripts

json

```
{
  "scripts": {
    "db:merge": "tsx src/core/database/mergeSchema.ts",
    "db:generate": "npm run db:merge && prisma generate",
    "db:push": "npm run db:merge && prisma db push",
    "db:migrate": "npm run db:merge && prisma migrate dev",
    "db:migrate:deploy": "npm run db:merge && prisma migrate deploy",
    "db:seed": "tsx prisma/seed.ts",
    "db:reset": "npm run db:merge && prisma migrate reset --force",
    "db:studio": "prisma studio"
  }
}
```

---

18. SEED DATA STRATEGY

Seed Components

1. System Configuration (20+ entries)

typescript

```
// Voting thresholds
{ key: 'voting.quorum.community', value: 0.40 }
{ key: 'voting.approval.major', value: 0.60 }

// Economic settings
{ key: 'dues.tier.ordinary', value: 60 }
{ key: 'ip.decay.monthly_rate', value: 0.10 }
{ key: 'pr.regeneration.monthly', value: 25 }

// Limits
{ key: 'marketplace.max_industries_per_user', value: 3 }
{ key: 'proposal.max_active_per_user', value: 5 }
```

---

2. Geographic Data (1,787 records)

```
Counties: 47
```

```
Constituencies: 290
Wards: 1,450
```

Data Source: Official IEBC data

Format: CSV files
```
counties.csv
constituencies.csv
wards.csv
```

---

3. System Groups (1,788 groups)
```
National: 1
Counties: 47
Constituencies: 290
Wards: 1,450
```

Created automatically from geographic data.

---

4. Roles & Permissions (10-15 roles)

typescript

```typescript
const SYSTEM_ROLES = [
  {
    name: 'system:super_admin',
    permissions: ['*:*']
  },
  {
    name: 'location:ward_admin',
    permissions: [
      'proposal:moderate:ward',
      'residence_request:review',
      'user:verify:community'
    ]
  },
  // ... more roles
```

```
];
```

---

5. Industries & Goods/Services (100+ items)

typescript

```typescript
const INDUSTRIES = [
  { name: 'Agriculture' },
  { name: 'Construction' },
  { name: 'Technology' },
  { name: 'Healthcare' },
  { name: 'Education' },
  { name: 'Transportation' },
  { name: 'Retail' },
  { name: 'Hospitality' },
  { name: 'Manufacturing' },
  { name: 'Services' },
];

const GOODS_SERVICES = [
  // Agriculture
  { name: 'Fresh Vegetables', industryId: 'agriculture', category: 'Produce' },
  { name: 'Livestock', industryId: 'agriculture', category: 'Animals' },
  { name: 'Farm Tools', industryId: 'agriculture', category: 'Equipment' },

  // Construction
  { name: 'Cement', industryId: 'construction', category: 'Materials' },
  { name: 'Labor Services', industryId: 'construction', category: 'Services' },

  // ... ~100 total
];
```

---

6. Onboarding Tutorials (10-15 tutorials)

typescript

```typescript
const TUTORIALS = [
  {
    key: 'platform_intro',
    title: 'Welcome to UjamaaDAO',
```

```
    category: 'BASICS',
    ipReward: 25,
    prReward: 10,
    estimatedMinutes: 5,
    content: { /* steps */ }
  },
  {
    key: 'governance_basics',
    title: 'How Governance Works',
    category: 'GOVERNANCE',
    ipReward: 25,
    prReward: 10,
    requiredFor: 'VOTING',
    estimatedMinutes: 10,
    content: { /* steps */ }
  },
  // ... more tutorials
];
```

---

7. Test Users (Optional - DEV only)

typescript

```
if (process.env.NODE_ENV === 'development') {
  const testUsers = [
    {
      email: 'admin@ujamaa.test',
      name: 'System Admin',
      roles: ['system:super_admin'],
      primaryWardId: kibera.id,
      globalImpactPoints: 10000,
      participationRights: 500,
    },
    {
      email: 'user@ujamaa.test',
      name: 'Test User',
      primaryWardId: kibera.id,
      globalImpactPoints: 500,
      participationRights: 100,
    },
```

```
    // ... more test users
  ];
}
```

Seed Execution Order

```
1. System Configuration
2. Industries & Goods/Services
3. Counties
4. Constituencies
5. Wards
6. System Groups (ward, constituency, county, national)
7. Roles & Permissions
8. Onboarding Tutorials
9. Test Users (dev only)
```

Important: Order matters due to foreign key constraints.

Seed Script Structure

typescript

```
// prisma/seed.ts

async function main() {
  console.log('🌱 Seeding database...');

  // 1. System Config
  await seedSystemConfiguration();

  // 2. Industries
  const industries = await seedIndustries();
  await seedGoodsServices(industries);

  // 3. Geography
  const counties = await seedCounties();
  const constituencies = await seedConstituencies(counties);
  const wards = await seedWards(constituencies);
```

```
  // 4. System Groups
  await seedSystemGroups(wards, constituencies, counties);

  // 5. RBAC
  await seedRoles();

  // 6. Tutorials
  await seedOnboardingTutorials();

  // 7. Test data (dev only)
  if (process.env.SEED_TEST_DATA === 'true') {
    await seedTestUsers(wards);
    await seedTestProposals();
  }

  console.log('    Seeding complete!');
}

main()
  .catch((e) => {
    console.error(e);
    process.exit(1);
  })
  .finally(async () => {
    await prisma.$disconnect();
  });
```

---

19. MIGRATION PLAN

From Current Schema to Final Schema

Step 1: Backup Current Database

bash

```
# Create backup
pg_dump -U ujamaa_user -h localhost ujamaa_db > backup_$(date +%Y%m%d_%H%M%S).sql

# Test restore (on test DB)
```

```
psql -U ujamaa_user -h localhost ujamaa_test_db < backup_20251212_143022.sql
```

Step 2: Generate Merged Schema

bash

```
# Merge all module schemas
npm run db:merge

# Review generated schema
cat prisma/schema.prisma

# Check for errors
npx prisma validate
```

Step 3: Create Migration

bash

```
# Generate migration (reviews changes)
npx prisma migrate dev --name complete_schema_v1 --create-only

# Review migration SQL
cat prisma/migrations/XXXXXX_complete_schema_v1/migration.sql

# Edit if needed (add data migrations, fix issues)
```

Step 4: Test Migration

bash

```
# Apply to test database
DATABASE_URL=$TEST_DATABASE_URL npx prisma migrate deploy

# Run seed on test DB
DATABASE_URL=$TEST_DATABASE_URL npm run db:seed

# Run tests
npm test

# Verify data integrity
npm run db:studio
```

Step 5: Deploy to Production

bash

```
# Create maintenance window
# Put app in read-only mode

# Backup production DB again
pg_dump -U ujamaa_user -h prod.db.host ujamaa_prod > prod_backup_final.sql

# Apply migration
DATABASE_URL=$PROD_DATABASE_URL npx prisma migrate deploy

# Run production seeds (system config, geo data)
DATABASE_URL=$PROD_DATABASE_URL npm run db:seed

# Verify
DATABASE_URL=$PROD_DATABASE_URL npx prisma studio

# Re-enable app
# Monitor for errors
```

Step 6: Data Migration Scripts

If you have existing data:

typescript

```
// prisma/migrations/XXXXXX_migrate_existing_data.ts

export async function migrateExistingUsers() {
  // Migrate old User.industryId to UserIndustry (many-to-many)
  const users = await prisma.user.findMany({
    where: { industryId: { not: null } }
  });

  for (const user of users) {
    await prisma.userIndustry.create({
      data: {
        userId: user.id,
        industryId: user.industryId,
        isPrimary: true,
```

```
        }
      });
    }

    // Remove old field (done in schema migration)
}

export async function createSystemGroups() {
    // Auto-create system groups for all wards
    const wards = await prisma.ward.findMany({
      include: { constituency: { include: { county: true } } }
    });

    for (const ward of wards) {
      // Create ward group
      await prisma.group.create({
        data: {
          name: `${ward.name} Ward Community`,
          isSystemGroup: true,
          systemType: 'WARD',
          locationScope: 'WARD',
          wardId: ward.id,
          constituencyId: ward.constituencyId,
          countyId: ward.constituency.countyId,
          status: 'ACTIVE',
          canBeDeleted: false,
          canBeRenamed: false,
        }
      });
    }
}

export async function autoEnrollExistingUsers() {
    // Enroll all existing users in their system groups
    const users = await prisma.user.findMany({
      where: { primaryWardId: { not: null } }
    });

    for (const user of users) {
      await enrollUserInSystemGroups(user.id, user.primaryWardId);
```

```
  }
}
```

Rollback Plan

bash

```
# If migration fails:

# 1. Restore from backup
psql -U ujamaa_user -h prod.db.host ujamaa_prod < prod_backup_final.sql

# 2. Revert code deployment
git revert <commit-hash>
npm run deploy

# 3. Investigate issue
# 4. Fix and retry
```

---

20. SERVICE LAYER CHANGES

Required Updates to Existing Services

auth.service.ts

Add:

typescript

```
// After email verification
await prisma.onboardingProgress.upsert({
  where: { userId: user.id },
  update: { emailVerified: true },
  create: {
    userId: user.id,
    emailVerified: true,
    currentStep: 'PROFILE_COMPLETION',
  },
});

// After wallet connection
```

```
await participationRightsService.awardPR(userId, 100, 'WALLET_CONNECTED');
await prisma.onboardingProgress.update({
  where: { userId },
  data: { walletConnected: true },
});

// Auto-enroll in system groups
await enrollUserInSystemGroups(userId, primaryWardId);
```

---

user.service.ts

Add:

typescript

```
// When user completes profile
await prisma.onboardingProgress.update({
  where: { userId },
  data: { profileCompleted: true },
});

// When user selects industries
await selectUserIndustries(userId, industryIds, primaryIndustryId);
await prisma.onboardingProgress.update({
  where: { userId },
  data: { industriesSelected: true },
});

// Check-in now awards location-specific IP
await locationImpactService.awardWardPoints(userId, wardId, 25, 'TEMPORARY_CHECKIN');
```

New Services Needed

1. **participationRights.service.ts**
   - `awardPR(userId, amount, reason)`
   - `spendPR(userId, amount, reason)`
   - `checkPRRequirement(userId, required)`
   - `monthlyRegeneration()` (cron)
   - `applyDuesPenalties()` (cron)
2. **locationImpact.service.ts**
   - `awardWardPoints(userId, wardId, amount, reason)`

- `awardGlobalPoints(userId, amount, reason)`
- `getUserImpactBreakdown(userId)`
- `getPrimaryWardImpact(userId)`
- `decayPoints()` (cron)

3. **onboarding.service.ts**
   - `getProgress(userId)`
   - `completeStep(userId, step)`
   - `isFeatureUnlocked(userId, feature)`
   - `recordMilestone(userId, key, ip, pr)`
4. **systemGroup.service.ts**
   - `enrollUserInSystemGroups(userId, wardId)`
   - `updateSystemGroupMemberships(userId, oldWard, newWard)`
   - `getSystemGroup(level, locationId)`
5. **dues.service.ts**
   - `recordPayment(userId, amount, tier, treasuryLevel)`
   - `checkOverdueDues()` (cron)
   - `applyPenalties(userId)`
   - `sendDuesReminders()` (cron)

---

21. SECURITY & ANTI-ABUSE

Security Measures

1. Rate Limiting

typescript

```typescript
const RATE_LIMITS = {
  // Authentication
  'POST /auth/signup': { max: 5, window: '1h' },
  'POST /auth/request-magic-link': { max: 3, window: '15m' },
  'POST /auth/connect-wallet': { max: 5, window: '15m' },

  // Proposals
  'POST /proposals': { max: 5, window: '1d' },
  'POST /proposals/:id/vote': { max: 100, window: '1d' },

  // Financial
  'POST /wallet/deposit': { max: 10, window: '1h' },
  'POST /wallet/withdraw': { max: 5, window: '1d' },
```

```
  // General
  'ALL /api/*': { max: 100, window: '15m' },
};
```

---

2. Input Validation

Use Zod schemas:

typescript

```
import { z } from 'zod';

const CreateProposalSchema = z.object({
  title: z.string().min(10).max(200),
  description: z.string().min(50).max(5000),
  proposalType: z.enum(['COMMUNITY_INITIATIVE', 'MAJOR_PROJECT', 'STRATEGIC_DECISION',
'EMERGENCY']),
  budget: z.number().positive().optional(),
  timeline: z.object({
    startDate: z.string().datetime(),
    endDate: z.string().datetime(),
  }).optional(),
});

// Middleware
router.post('/proposals',
  authenticate,
  validate(CreateProposalSchema),
  proposalController.create
);
```

---

3. Treasury Security

- **Multi-sig for large amounts** (> KES 50,000)
- **Spending thresholds** per group type
- **Audit logs** for all transactions
- **Delayed execution** for large proposals (24hr hold)
- **Emergency freeze** mechanism (admin can pause)

---

4. Wallet Security

- **Signature verification** (ethers.js)
- **Nonce rotation** (prevent replay attacks)
- **Recovery wallet** (backup access)
- **Session tokens** (not private keys)
- **Hardware wallet support** (future)

---

5. Data Encryption

- **At rest:** AES-256 for sensitive fields
- **In transit:** TLS 1.3 (HTTPS everywhere)
- **Passwords:** Bcrypt (if we add passwords later)
- **API keys:** Stored hashed

---

6. Abuse Prevention

Attack Vector	Mitigation
Sybil attacks	Email + phone verification, PR costs
Vote manipulation	IP-based weighting, verification requirements
Proposal spam	PR costs, daily limits, verification tiers
Treasury drain	Multi-sig, spending limits, audit trails
Marketplace fraud	Escrow, reviews, dispute resolution
False emergencies	Penalties, verification, community flagging

---

22. TESTING STRATEGY

Test Types

1. Unit Tests (Vitest/Jest)

Test individual functions in isolation:

typescript

```
describe('participationRightsService', () => {
  describe('awardPR', () => {
    it('should increase user PR balance', async () => {
      const userId = 'test-user-id';
      await participationRightsService.awardPR(userId, 100, 'DUES_PAID');

      const user = await prisma.user.findUnique({ where: { id: userId } });
      expect(user.participationRights).toBe(100);
```

```
    });

    it('should create PR log entry', async () => {
      const userId = 'test-user-id';
      await participationRightsService.awardPR(userId, 100, 'DUES_PAID');

      const logs = await prisma.participationRightsLog.findMany({ where: { userId } });
      expect(logs).toHaveLength(1);
      expect(logs[0].amount).toBe(100);
      expect(logs[0].reason).toBe('DUES_PAID');
    });
  });
});
```

---

2. Integration Tests (with Test DB)

Test API endpoints end-to-end:

typescript

```
describe('POST /proposals', () => {
  let testUser;
  let testWard;
  let authToken;

  beforeAll(async () => {
    // Setup test DB
    testWard = await createTestWard();
    testUser = await createTestUser({ primaryWardId: testWard.id });
    authToken = await generateAuthToken(testUser.id);
  });

  it('should create ward proposal with sufficient PR', async () => {
    // Give user enough PR
    await prisma.user.update({
      where: { id: testUser.id },
      data: { participationRights: 100 }
    });

    const response = await request(app)
      .post('/api/proposals')
```

```
        .set('Authorization', `Bearer ${authToken}`)
        .send({
          title: 'Fix Ward Road',
          description: 'Road needs repair urgently...',
          proposalType: 'COMMUNITY_INITIATIVE',
          budget: 50000,
        });

      expect(response.status).toBe(201);
      expect(response.body.proposal.title).toBe('Fix Ward Road');

      // Verify PR was spent
      const updatedUser = await prisma.user.findUnique({ where: { id: testUser.id } });
      expect(updatedUser.participationRights).toBe(50); // 100 - 50 cost
    });

    it('should fail if insufficient PR', async () => {
      await prisma.user.update({
        where: { id: testUser.id }, data: { participationRights: 10 } // Not enough });

const response = await request(app)
    .post('/api/proposals')
    .set('Authorization', `Bearer ${authToken}`)
    .send({
      title: 'Fix Ward Road',
      description: 'Road needs repair urgently...',
      proposalType: 'COMMUNITY_INITIATIVE',
    });

expect(response.status).toBe(403);
expect(response.body.error).toContain('Insufficient participation rights');

}); });

---

### **3. Smart Contract Tests** (Hardhat)

Test blockchain integration:
```typescript
describe('UjamaaToken', () => {
```
```

```
  let token;
  let owner;
  let treasury;

  beforeEach(async () => {
    [owner, treasury] = await ethers.getSigners();
    const Token = await ethers.getContractFactory('UjamaaToken');
    token = await Token.deploy(treasury.address);
  });

  it('should mint tokens to treasury', async () => {
    const balance = await token.balanceOf(treasury.address);
    expect(balance).to.equal(ethers.parseEther('1000000')); // 1M tokens
  });

  it('should allow treasury to transfer', async () => {
    await token.connect(treasury).transfer(owner.address, ethers.parseEther('100'));
    const balance = await token.balanceOf(owner.address);
    expect(balance).to.equal(ethers.parseEther('100'));
  });
});
```

---

4. E2E Tests (Playwright/Cypress)

Test complete user flows:
```typescript
test('complete user onboarding flow', async ({ page }) => {
  // 1. Sign up
  await page.goto('/signup');
  await page.fill('[name="email"]', 'test@example.com');
  await page.fill('[name="name"]', 'Test User');
  await page.selectOption('[name="ward"]', 'kibera');
  await page.click('button[type="submit"]');

  // 2. Verify email (simulate)
  const token = await getVerificationToken('test@example.com');
  await page.goto(`/verify-email?token=${token}`);
```

```
    await expect(page).toHaveURL('/dashboard');

    // 3. Select industries
    await page.goto('/onboarding/industries');
    await page.check('[value="agriculture"]');
    await page.check('[value="technology"]');
    await page.click('button:text("Continue")');

    // 4. Complete platform intro
    await page.goto('/onboarding/tutorials');
    await page.click('text=Platform Introduction');
    // ... complete tutorial steps
    await page.click('button:text("Complete Tutorial")');

    // 5. Verify onboarding progress
    await page.goto('/dashboard');
    await expect(page.locator('.onboarding-progress')).toContainText('60%');
});
```

---

5. Load Testing (k6/Artillery)

Test system under load:
```javascript
import http from 'k6/http';
import { check, sleep } from 'k6';

export let options = {
    vus: 100, // 100 virtual users
    duration: '5m',
};

export default function () {
    // Login
    let loginRes = http.post('https://api.ujamaa.test/auth/login', {
        email: 'user@test.com',
        password: 'test123',
    });
```

```
    check(loginRes, {
        'login successful': (r) => r.status === 200,
    });

    let token = loginRes.json('accessToken');

    // Get proposals
    let proposalsRes = http.get('https://api.ujamaa.test/proposals', {
        headers: { Authorization: `Bearer ${token}` },
    });

    check(proposalsRes, {
        'proposals loaded': (r) => r.status === 200,
        'response time < 500ms': (r) => r.timings.duration < 500,
    });

    sleep(1);
}
```

Test Coverage Goals

Component	Target Coverage
Services	90%+
Controllers	85%+
Utils	95%+
Middleware	90%+
Smart Contracts	100%

CI/CD Pipeline

```yaml
# .github/workflows/test.yml

name: Test Suite

on: [push, pull_request]
```

```yaml
jobs:
  test:
    runs-on: ubuntu-latest

    services:
      postgres:
        image: postgres:15
        env:
          POSTGRES_DB: ujamaa_test
          POSTGRES_USER: test_user
          POSTGRES_PASSWORD: test_pass
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5

    steps:
      - uses: actions/checkout@v3

      - uses: actions/setup-node@v3
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Merge schemas
        run: npm run db:merge

      - name: Generate Prisma client
        run: npx prisma generate

      - name: Run migrations
        run: npx prisma migrate deploy
        env:
          DATABASE_URL: postgresql://test_user:test_pass@localhost:5432/ujamaa_test

      - name: Run unit tests
```

```
            run: npm run test:unit

          - name: Run integration tests
            run: npm run test:integration
            env:
              DATABASE_URL: postgresql://test_user:test_pass@localhost:5432/ujamaa_test

          - name: Upload coverage
            uses: codecov/codecov-action@v3
```

---

23. DEVOPS & INFRASTRUCTURE

Docker Setup

docker-compose.yml
```yaml
version: '3.8'

services:
  postgres:
    image: postgres:15-alpine
    container_name: ujamaa_postgres
    environment:
      POSTGRES_USER: ujamaa_user
      POSTGRES_PASSWORD: ujamaa_pass
      POSTGRES_DB: ujamaa_db
    volumes:
      - postgres_data:/var/lib/postgresql/data
    ports:
      - "5432:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ujamaa_user"]
      interval: 10s
      timeout: 5s
      retries: 5

  postgres_test:
```

```
    image: postgres:15-alpine
    container_name: ujamaa_postgres_test
    environment:
      POSTGRES_USER: ujamaa_user
      POSTGRES_PASSWORD: ujamaa_pass
      POSTGRES_DB: ujamaa_test_db
    ports:
      - "5433:5432"
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U ujamaa_user"]
      interval: 10s
      timeout: 5s
      retries: 5

  backend:
    build:
      context: ./backend
      dockerfile: Dockerfile
    container_name: ujamaa_backend
    depends_on:
      postgres:
        condition: service_healthy
    environment:
      NODE_ENV: development
      DATABASE_URL: postgresql://ujamaa_user:ujamaa_pass@postgres:5432/ujamaa_db
      TEST_DATABASE_URL:
postgresql://ujamaa_user:ujamaa_pass@postgres_test:5432/ujamaa_test_db
      JWT_SECRET: ${JWT_SECRET}
      FRONTEND_URL: http://localhost:3000
    volumes:
      - ./backend:/usr/src/app
      - /usr/src/app/node_modules
    ports:
      - "4000:4000"
    command: npm run dev

  blockchain:
    build:
      context: ./blockchain
      dockerfile: Dockerfile
```

```
    container_name: ujamaa_blockchain
    ports:
      - "8545:8545"
    command: npx hardhat node

volumes:
  postgres_data:
```

Dockerfile (Backend)
```dockerfile
FROM node:20-alpine AS builder

WORKDIR /usr/src/app

COPY package*.json ./
RUN npm ci

COPY . .

# Merge schemas
RUN npm run db:merge

# Generate Prisma client
RUN npx prisma generate

# Build TypeScript
RUN npm run build

# Production image
FROM node:20-alpine

WORKDIR /usr/src/app

COPY package*.json ./
RUN npm ci --only=production

COPY --from=builder /usr/src/app/dist ./dist
COPY --from=builder /usr/src/app/prisma ./prisma
COPY --from=builder /usr/src/app/node_modules/.prisma ./node_modules/.prisma
```

```
EXPOSE 4000

CMD ["node", "dist/index.js"]
```

Deployment Strategy

Environments:

1. **Development** - Local (Docker Compose)
2. **Staging** - Cloud (test with prod-like data)
3. **Production** - Cloud (live)

Deployment Flow:

1. Developer commits to feature branch └─ CI runs tests
2. PR opened to `develop` ├─ Code review ├─ Automated tests └─ Preview deployment (optional)
3. PR merged to `develop` └─ Auto-deploy to Staging
4. Staging testing ├─ QA team tests ├─ Load testing └─ Security scan
5. Release created (tag v1.2.3) └─ Deploy to Production
6. Monitor ├─ Error tracking (Sentry) ├─ Performance (New Relic) └─ Logs (CloudWatch)

Monitoring & Alerting

Key Metrics:
```typescript
// Health check endpoint
app.get('/health', async (req, res) => {
  try {
    // Check DB
    await prisma.$queryRaw`SELECT 1`;

    // Check Redis (if used)
    // await redis.ping();

    res.json({
```

```
        status: 'healthy',
        timestamp: new Date(),
        uptime: process.uptime(),
        database: 'connected',
      });
    } catch (error) {
      res.status(503).json({
        status: 'unhealthy',
        error: error.message,
      });
    }
});
```

Alerts:

- Error rate > 1%
- Response time > 1s (P95)
- Database connections > 80%
- Disk usage > 85%
- Failed logins spike
- Treasury transaction anomalies

---

24. CONSTANTS & THRESHOLDS

Default Values (SystemConfiguration)

Voting Thresholds:
```typescript
const VOTING_THRESHOLDS = {
  COMMUNITY_INITIATIVE: {
    quorum: 0.40,        // 40% of eligible voters
    approval: 0.50,      // 50% of votes cast
    votingPeriod: 7,     // days
  },
  MAJOR_PROJECT: {
    quorum: 0.50,
    approval: 0.60,
    votingPeriod: 14,
```

```
  },
  STRATEGIC_DECISION: {
    quorum: 0.60,
    approval: 0.66,
    votingPeriod: 21,
  },
  EMERGENCY: {
    quorum: 0.30,
    approval: 0.60,
    votingPeriod: 3,
  },
};
```

Economic Settings:
```typescript
const ECONOMIC_SETTINGS = {
  // Dues
  DUES_TIERS: {
    ORDINARY: 60,    // KES/month
    SUPPORTER: 200,
    SPONSOR: 1000,
  },

  // Impact Points
  IP_DECAY_RATE: 0.10,              // 10% monthly
  IP_DECAY_ACTIVE_USER: 0.05,       // 5% for active users
  IP_GRACE_PERIOD_MONTHS: 3,        // No decay first 3 months

  // Participation Rights
  PR_MONTHLY_REGEN: 25,             // Base regeneration
  PR_MAX_BALANCE: 500,              // Cap
  PR_LOW_WARNING: 20,               // Show warning

  // Marketplace
  MAX_INDUSTRIES_PER_USER: 3,
  LISTING_EXPIRY_DAYS: 90,
```

```
  // Treasury
  TREASURY_SPENDING_LIMITS: {
    WARD: { small: 10000, medium: 100000 },
    CONSTITUENCY: { small: 50000, medium: 500000 },
    COUNTY: { small: 100000, medium: 1000000 },
    NATIONAL: { small: 0, medium: 0 },  // All via proposal
  },
};
```

Verification Requirements:
```typescript
const VERIFICATION_REQUIREMENTS = {
  // Proposal creation
  PROPOSAL_CREATE: {
    WARD: {
      minVerification: 'PHONE_VERIFIED',
      minIP: 100,
      prCost: 50,
    },
    CONSTITUENCY: {
      minVerification: 'COMMUNITY_VERIFIED',
      minIP: 500,
      prCost: 100,
    },
    COUNTY: {
      minVerification: 'COMMUNITY_VERIFIED',
      minIP: 500,
      prCost: 150,
    },
    NATIONAL: {
      minVerification: 'COMMUNITY_VERIFIED',
      minIP: 1000,
      prCost: 200,
    },
  },

  // Feature access
```

```
  VOTING: {
    minVerification: 'EMAIL_VERIFIED',
    paidDues: true,
    minPR: 5,
  },

  MARKETPLACE_SELL: {
    minVerification: 'PHONE_VERIFIED',
    industriesSelected: true,
    minPR: 20,
  },

  GROUP_CREATE: {
    minVerification: 'COMMUNITY_VERIFIED',
    minIP: 200,
    minPR: 100,
  },

  MILESTONE_VERIFY: {
    minVerification: 'COMMUNITY_VERIFIED',
    minIP: 500,
    independent: true,  // Not project participant
  },
};
```

Work & IP Rates:
```typescript
const WORK_IP_RATES = {
  MANUAL_LABOR: 10,      // IP per hour
  SKILLED_WORK: 20,
  TECHNICAL_WORK: 30,
  SUPERVISION: 15,

  QUALITY_MULTIPLIERS: {
    POOR: 0.5,
    ACCEPTABLE: 1.0,
    GOOD: 1.5,
```

```
    EXCELLENT: 2.0,
  },

  DAILY_CAP: 100,        // Max IP per day from work
};
```

Rate Limits:
```typescript
const RATE_LIMITS = {
  AUTH_SIGNUP: { max: 5, window: '1h' },
  AUTH_MAGIC_LINK: { max: 3, window: '15m' },
  PROPOSAL_CREATE: { max: 5, window: '1d' },
  VOTE: { max: 100, window: '1d' },
  WALLET_DEPOSIT: { max: 10, window: '1h' },
  WALLET_WITHDRAW: { max: 5, window: '1d' },
  API_GENERAL: { max: 100, window: '15m' },
};
```

---

25. IMPLEMENTATION ROADMAP

Phased Development Plan

Phase 1: Foundation (Weeks 1-4)

Goal: Basic platform with auth, users, groups

Week	Deliverables
Week 1	Final schema merged Database migrations Seed scripts
Week 2	Auth system (magic link) User management Profile/industries
Week 3	System groups created Auto-enrollment Group membership
Week 4	Basic UI (dashboard, profile) Integration tests Deploy to staging

MVP Features:
- Email signup + magic link login
- User profiles with industries
- Auto-join 4 system groups
- Basic group viewing

Phase 2: Onboarding & PR (Weeks 5-6)

Goal: Tutorial system + Participation Rights

Week	Deliverables
Week 5	Onboarding system Tutorial CMS Progress tracking Milestone rewards
Week 6	PR service layer Dues payment integration PR earning/spending Penalties system

MVP Features:
- Step-by-step onboarding with progress bar
- 5 core tutorials (platform intro, governance, etc.)
- PR tracking and enforcement
- Mpesa dues payment

Phase 3: Governance (Weeks 7-9)

Goal: Proposals, voting, IP weighting

Week	Deliverables
Week 7	Proposal creation Scope validation PR cost enforcement Draft/publish flow
Week 8	Voting system IP weight calculation Location-specific boosts Quorum/approval logic
Week 9	Vote tracking Proposal status updates Results display E2E tests

MVP Features:
- Create ward-level proposals

- Vote with IP-weighted votes
- Real-time vote tallying
- Auto-approve when thresholds met

Phase 4: Projects & Work (Weeks 10-12)

Goal: Project execution, milestones, physical work tracking

Week	Deliverables
Week 10	Project creation from proposals Milestone definition Budget allocation Verifier assignment
Week 11	Physical work logs Photo upload Witness tracking Verification flow
Week 12	Milestone approval Fund release IP awards Project completion

MVP Features:
- Convert approved proposal → project
- Log physical work with photos
- Verify milestones
- Earn IP from work

Phase 5: Marketplace (Weeks 13-14)

Goal: Basic marketplace with trading rules

Week	Deliverables
Week 13	Listing creation Industry validation Seller type restrictions Search/browse
Week 14	Transaction flow Escrow system Reviews Dispute resolution (basic)

MVP Features:
- List goods/services
- Buy from groups only (individuals)
- Escrow-protected payments

- Leave reviews

Phase 6: Education & Emergency (Weeks 15-16)

Goal: Learning content + emergency alerts

Week	Deliverables
Week 15	Educational modules Assessments Completion tracking IP rewards
Week 16	Emergency alerts Volunteer response Resource coordination Fast-track funding

MVP Features:
- 10+ educational modules
- Earn IP from learning
- Report emergencies
- Coordinate response

Phase 7: Treasury & Economics (Weeks 17-18)

Goal: Full economic system + wallet integration

Week	Deliverables
Week 17	Treasury management UI Multi-sig for large amounts Transaction history Monthly reports
Week 18	UT token integration Mpesa withdrawals Wallet balance tracking Exchange rate management

MVP Features:
- View group treasury
- Approve spending
- Withdraw funds via Mpesa
- Track all transactions

Phase 8: Polish & Launch (Weeks 19-20)

Goal: Bug fixes, optimization, production-ready

Week	Deliverables
Week 19	Performance optimization Security audit Load testing Bug fixes
Week 20	Production deployment User documentation Admin training **PUBLIC LAUNCH**

Post-Launch Priorities:

Month 2:
- Wallet connection (blockchain)
- Smart contract integration
- Location verification system
- Residence change approvals

Month 3:
- Mobile app (React Native)
- Advanced analytics dashboard
- Conflict resolution enhancements
- Community verification expansion

Month 4:
- Banking integrations
- International payments (diaspora)
- Advanced marketplace features
- Educational content expansion

Month 5:
- Governance improvements based on feedback
- Accessibility enhancements
- Language support (Swahili full translation)
- Performance optimizations

Month 6+:
- Feature requests from community
- Regional expansion

- Partnership integrations
- Platform sustainability features

Success Metrics

Launch Goals (Month 1):
- 1,000+ registered users
- 100+ active groups
- 50+ proposals created
- 10+ projects funded
- KES 1M+ in treasuries
- 90%+ uptime

3-Month Goals:
- 10,000+ users
- 500+ groups
- 200+ completed projects
- KES 10M+ in treasuries
- 95%+ uptime
- < 2s average response time

6-Month Goals:
- 50,000+ users
- 2,000+ groups
- 1,000+ completed projects
- KES 50M+ in treasuries
- Profitable/sustainable
- Expand to 10+ counties

Documentation Deliverables

Technical Docs:
1. API Documentation (OpenAPI/Swagger)
2. Database Schema Documentation
3. Smart Contract Documentation
4. Deployment Guide
5. Development Setup Guide

User Docs:

1. User Guide (How to use platform)
2. Group Admin Guide
3. Verifier Handbook
4. FAQ
5. Video Tutorials

Governance Docs:

1. Platform Constitution
2. Governance Rules
3. Voting Guidelines
4. Treasury Management Policy
5. Dispute Resolution Process

CONCLUSION

This document represents the **complete updated business logic** for UJAMAADAO incorporating all decisions made during our schema audit and architecture review.